

Cooperative defense of autonomous surface vessels with quantity disadvantage using behavior cloning and deep reinforcement learning

Siqing Sun ^{a,b}, Tianbo Li ^{a,*}, Xiao Chen ^c, Huachao Dong ^{a,b}, Xinjing Wang ^{a,b}

^a School of Marine Science and Technology, Northwestern Polytechnical University, Xi'an 710072, China

^b Aoxiang State Key Laboratory, Northwestern Polytechnical University, Xi'an 710072, China

^c Air and Missile Defense College, Air Force Engineering University, Xi'an 710051, China

ARTICLE INFO

Keywords:

Autonomous surface vessels
Cooperative defense
Behavior cloning
Deep reinforcement learning
Reward design

ABSTRACT

Autonomous Surface Vessels (ASVs) excel at undertaking hazardous tasks, garnering significant attention recently. Particularly, ASV cooperative defense is a crucial application for protecting harbors and combating smugglers. Here, ASVs intercept intruders from reaching a protected region. Unlike most research, which assumes defenders with numerical advantages, this work considers a more practical defense mission with fewer defenders, defender damages, and intruders employing evasion strategies. However, interception challenges are also introduced, including ASV underactuated dynamics, a limited interception time window, and environmental nonstationarity. Directly applying existing defense methods to such missions may not achieve success. To surmount the challenges, we propose an ASV decision-making framework by integrating supervised learning and deep reinforcement learning. Initially, supervised learning uses actions from a bi-level controller to train ASVs, addressing underactuated dynamics and aiding policy convergence. Subsequently, deep reinforcement learning explores more effective policies to enhance interception rates. Furthermore, hybrid rewards are meticulously designed to drive policy optimizations while mitigating environmental nonstationarity. Finally, numerical simulations are carried out to verify the effectiveness of our approach.

1. Introduction

Utilizing ASVs for complex missions offers significant advantages [1], including risk mitigation for human casualties and low-cost mission execution. Notably, the task of safeguarding a specific asset or region has garnered significant attention recently [2]. Lowe et al. [3] explore predator cooperation strategies to prevent prey from reaching their food sources. Similarly, Raboin et al. [4] appoint a team of ASVs to guard an asset in an environment characterized by hostile boats and civilian traffic, highlighting the greater application potential of cooperative defense over individual interception. For other unmanned systems, Meng et al. [5] control autonomous underwater vehicles for cooperative harbor protection. Yu et al. [6] dispatched drones to intercept hostile drones, so as to safeguard a military base. Tan et al. [7] guide multiple missiles to collaboratively intercept a target with desired impact angles. These works reveal that cooperative defense is a multifaceted mission with diverse research priorities. For example, some studies focus on addressing the challenges of controlling precision in unmanned systems [8], which often simplifies task requirements to protect a point rather than an entire area. Kang et al. [9]

explore a domain protection scenario of ASVs in an obstructive environment. However, critical factors are overlooked, such as asymmetrical capabilities between defenders and intruders, and potential collision damages among allied ASVs, reducing the applicability. To consider more practical elements, this paper introduces an ASV cooperative defense scenario (ACDS), characterized by a fewer defenders, considerations for defender damages, and intruder evasion strategies towards defenders.

The key to cooperative defense lies in advanced autonomous decision-making capabilities [10–12]. There are three primary categories of decision-making techniques [13], including game-based [14], optimization-based [15], and learning-based [16]. Game-based methods utilize game theory to model protection processes and seek equilibriums such as Nash or Stackelberg equilibriums. Chen et al. [17] derive the equilibrium strategy of a three dimensional pursuit-evasion game, by solving the retrogressive path equation according to the Hamilton–Jacobi–Bellman–Isaacs. These methods excel at capturing precise solutions [18]. However, prior knowledge of opponent strategies are required in the methods to construct cost functions, limiting the

* Corresponding author.

E-mail addresses: sunsiqing@nwpu.edu.cn (S. Sun), litianbo@mail.nwpu.edu.cn (T. Li), qwfy628@mail.nwpu.edu.cn (X. Chen), hdong@nwpu.edu.cn (H. Dong), wangxinjing0213@nwpu.edu.cn (X. Wang).

<https://doi.org/10.1016/j.asoc.2024.111968>

Received 18 December 2023; Received in revised form 2 July 2024; Accepted 4 July 2024

Available online 9 July 2024

1568-4946/© 2024 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

applicability in real-world scenarios [19]. Optimization-based methods [20] aim to find optimal actions for task planning problems, often employing evolutionary algorithms or other metaheuristics [21]. Li et al. [22] focus on patrol missions of multiple ocean points, and combine simulated annealing and genetic algorithms to determine optimal actions for task assignment and path planning. Metaheuristics offer advantages in decision space exploration and exhibit stable convergence properties [23], but they suffer from slow optimization processes, facing challenges in real-time decision-making [24,25]. Learning-based methods, specifically deep reinforcement learning [26], have demonstrated remarkable performance across various domains, including Atari video games, Go, and StarCraft [27–29]. Learning-based methods do not rely on dynamic models or prior knowledge, and can promptly output actions through policy neural networks. These characteristics demonstrate significant application potential in cooperative defense [30]. Li et al. [31] adopt deep reinforcement learning to improve the efficiency of multi-drones rounding up a maneuvering target. Nonetheless, the system in ACDS switches constantly due to agent damages, resulting in dynamic and multimodal rewards. Additionally, the underactuated nature of ASVs complicates the association between rewards and actions. Consequently, learning-based methods may encounter sampling inefficiency and non-convergence in ACDS, which are central challenges to be addressed in this research.

Fortunately, supervised learning proves to be useful in accelerating and converging reinforcement learning [32], which adopts behavioral cloning to capture and replicate motor skills derived from expert demonstrations. Notably, Hester et al. [33] introduce the concept of deep Q-learning from expert demonstrations, which establishes a supervised loss through demonstration data to expedite the learning process. Subsequently, research has progressed towards integrating demonstrations with various reinforcement learning methods, gradually finding applications in diverse domains. For instance, Huang et al. [34] develop an actor-critic learning framework that combines imitative human priors, delivering superior performance compared to baseline algorithms in autonomous driving tasks. However, it is worth noting that these methods are predominantly designed for single-agent decision-making. When applied to multi-agent cooperation scenarios, due to the lack of information sharing among agents, they encounter a significant challenge in achieving effective collaboration [35]. To overcome this limitation, Lowe et al. [3] pioneered a framework that combines centralized training with decentralized execution, effectively extending Deep Deterministic Policy Gradient (DDPG) to Multi-Agent DDPG (MADDPG). MADDPG has since gained prominence as one of the leading methods in multi-agent defense problem, due to its remarkable performance [36,37]. Therefore, MADDPG is also adopted and modified with expert data in this work. Next, acquiring sufficient preferred demonstrations presents another challenge. Unlike autonomous driving vehicles that can reference general car data, it is much more difficult to obtain demonstrations of underactuated ASVs in a defense scenario [38]. Meanwhile, it is noticed that most learning-based methods are model-free algorithms [39], which neglect the deterministic information of ASV kinematics and dynamics, thereby hindering training efficiency. Thus, it is preferable to integrate demonstrations with the ASV information. Hence, we develop a bi-level controller to generate preferred actions for ASVs, where the upper level is a position controller, and the lower level is an attitude controller. The bi-level controller can follow the deterministic information to solve the underactuated problem, and effectively expand the datasets. In the end, we propose a defense method that combines MADDPG and behavioral cloning, incorporating the actions generated by the bi-level controller.

Once the algorithmic framework is established, designing reward functions becomes a pivotal factor in driving policy optimizations [40], as training processes commonly confront challenges associated with sparse rewards. Agents receive reinforcement feedback or rewards only upon achieving specific milestones or completing predefined objectives. The scarcity of rewards can complicate credit assignment, making it

challenging for agents to discern the impact of their actions on the overall task. Prior research in the domain of region protection [41–44] conventionally rely on distance-based shape rewards, which involve incorporating positive intruder-region distances and negative defender-intruder distances. Moreover, these studies typically introduce a substantial reward bonus upon completing an episode. These rewards can encourage defenders to actively pursue intruders, and meanwhile, prevent intruders from reaching the protected region. However, the rewards in the aforementioned ACDS often undergo dynamic transformations during operation, leading agents to become trapped in local optima. Consequently, the interception policy becomes inefficient, and ASVs fail to defend the area. To address this issue, we design innovative and meticulous reward functions from three distinct perspectives to enhance the overall system performance. In summary, this work has the following contributions:

1. Developing an ASV Cooperative Defense Method (ACDM) by combining behavior cloning and reinforcement learning, significantly improving protection success rates in high nonstationary environments with fewer defenders.
2. Proposing a bi-level controller to generate actions for addressing the underactuated nature of ASVs, so that ASVs can imitate the actions to accelerate the training convergence in behavior cloning.
3. Designing reward with hybrid terms to effectively drive policy optimizations while mitigating environmental nonstationarity, and verifying the effectiveness of ACDM by numerical simulations.

The remainder of this paper is organized as follows. Section 2 constructs the challenging region protection scenario ACDS. Section 3 develops a learning-based defense method ACDM, by combining behavior cloning and MADDPG. Section 4 conducts numerical simulations and comparisons, to substantiate the effectiveness of the proposed learning method. Conclusions are finally drawn, and future works are presented in Section 5.

数量劣势下ASV协同防御场景

2. Cooperative defense scene of ASVs with quantity disadvantage

Throughout the paper, $\mathbb{N}^+$ denotes the positive integer set. $\mathbb{R}$ and $\mathbb{R}^+$ denote the real number and positive real number, respectively. $\|*\|$ is the 2-norm of a vector $*$, $*^T$ represents the transpose of a matrix $*$, and $\mathbb{E}(*)$ is the expectation of $*$. Symbols p_i^t , v_i^t , and a_i^t are coordinate, velocity, and actions of agent i in x - and y -directions at moment t , respectively. $*_{i,j}^t$ denotes the relative value of $*$ between agents i and j at a temporal instant t .

In this section, we create a cooperative defense scenario for ASVs facing a quantity disadvantage. To begin, we introduce the underactuated kinetics and dynamics models of ASVs. Next, we outline the protection process and describe representative situations, such as intruder capture or arrival. Additionally, we lay out the criteria for determining the success or failure of defenders. In the end, we develop a bi-level control strategy for an ASV to generate actions, which can address the underactuated problem. The bi-level controller can guide an ASV to pursue a moving target while avoiding collisions, which enhances the intrusion efficiency and offers a reference for defender behavior cloning.

ASV的欠驱动动力学与动力学模型

2.1. Underactuated kinetics and dynamics models of ASVs

This section introduces underactuated kinetics and dynamics models of ASVs. Denote the position distribution of the ASVs as $P = [p_0, p_1, \dots, p_{m+n}]^T$, where $p_i = [x_i, y_i]^T \in \mathbb{R}^2$, $i \in 0, 1, \dots, N$ is the Cartesian coordinate of the i th vehicle. Besides, $[X_B, Y_B]$ is the hull

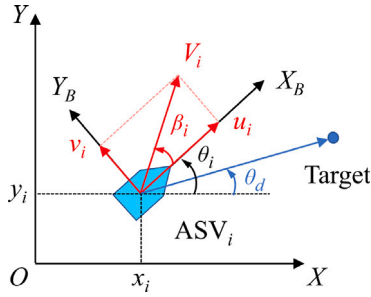


Fig. 1. An illustration of the kinematics of a vehicle. Here, u_i is the surge velocity, v_i is the sway velocity, θ_i is the heading angle, $V_i = \sqrt{u_i^2 + v_i^2}$ is the actual velocity and β_i is the drift angle and can be calculated by $\beta_i = \text{atan2}(v_i/\|u_i\|)$.

coordinate system. As shown in Fig. 1, the kinetics of an ASV can be expressed as follows

$$\begin{aligned}\dot{x}_i &= u_i \cos \theta_i - v_i \sin \theta_i, \\ \dot{y}_i &= u_i \sin \theta_i + v_i \cos \theta_i, \\ \dot{\theta}_i &= \omega_i, i \in \mathcal{V},\end{aligned}\quad (1)$$

where (x_i, y_i) denotes the position of the i th vessel, θ_i is the heading angle of the i th vessel in the earth reference, respectively. u_i, v_i, ω_i are the surge, sway, and yaw velocities of the i th vessel, respectively. Note that $V_i = \sqrt{u_i^2 + v_i^2}$ is the actual velocity and β_i is the drift angle and can be calculated by $\beta_i = \text{atan2}(v_i/\|u_i\|)$.

Assuming that the sideslip angle β_i is small, the dynamics of the i th ASV is defined as follows

$$\begin{aligned}\dot{u}_i &= \frac{m_i''}{m_i'} v_i \omega_i - \frac{d_i'}{m_i'} u_i + \frac{1}{m_i'} \tau_i^u, \\ \dot{\omega}_i &= \frac{m_i' - m_i''}{m_i'''} u_i v_i - \frac{d_i'''}{m_i'''} \omega_i + \frac{1}{m_i'''} \tau_i^\omega, \\ \dot{v}_i &= -\frac{m_i'}{m_i''} u_i \omega_i - \frac{d_i''}{m_i''} v_i,\end{aligned}\quad (2)$$

where m_i', m_i'', m_i''' are individual ASV inertia, d_i', d_i'', d_i''' are the hydrodynamic damping coefficients, and $\tau_i = [\tau_i^u, \tau_i^\omega]^T \in \mathbb{R}^2$ is the control input vector of i th vessel, comprises a surge force τ_i^u and a yaw moment τ_i^ω . Thus, the aim of this study is to utilize neural networks to generate τ_i^u and τ_i^ω for defenders, enabling autonomous decision-making.

2.2. Cooperative defense process 合作防御过程

As is shown in Fig. 2, we construct an ASV Cooperative Defense Scene (ACDS). In ACDS, there are two opponent groups of underactuated ASVs, i.e., m defenders $\mathcal{V}_d = \{\text{ASV}_1, \text{ASV}_2, \dots, \text{ASV}_m\}$, and n intruders $\mathcal{V}_r = \{\text{ASV}_{m+1}, \text{ASV}_{m+2}, \dots, \text{ASV}_{m+n}\}$ ($m < n, m, n \in \mathbb{N}^+$). The defenders $\mathcal{V}_d$ aim to intercept the intruders $\mathcal{V}_r$ from reaching a center circular region, which is defined by the symbol α . The coordinates and radius of the region α are represented as p_α and r_α . Besides, ACDS is constrained within a square with dimensions of $b_s \times b_s$. A complete defense process is defined as an episode, which starts with a time variable $t = 0$ and extends until the protection task is completed. An episode is outlined as follows.

(1) Initialization: ASVs are placed randomly in ACDS to ensure generality. Specifically, defenders are positioned at the center, while intruders are located at the edge of the ACDS. This is expressed as $0 < \|p_i^0 - p_\alpha\| < 0.3b_s, i \in \mathcal{V}_d, 0.8b_s < \|p_j^0 - p_\alpha\| < 0.95b_s, j \in \mathcal{V}_r$.

(2) Defense processes and constraints: ASVs move driven by Eqs. (1), (2). For each ASV, real-time surge force τ_i^u and yaw moment τ_i^ω cannot exceed the set maximum values $\tau_i^{\max} = [\tau_{\max}^u, \tau_{\max}^\omega] \in \mathbb{R}^+$, i.e., $0 < \tau_i^u < \tau_{\max}^u, |\tau_i^\omega| < \tau_{\max}^\omega$. Since defenders $\mathcal{V}_d$ face quantity disadvantage

($m < n$), they should possess greater mobility to carry out the protection task, i.e., $\tau_i^{\max} > \tau_j^{\max}, i \in \mathcal{V}_d, j \in \mathcal{V}_r$.

(3) Typical situations: As the protection process unfolds, four distinct situations can occur ($\Gamma \neq \emptyset$), as illustrated in Fig. 2(b), including Destroyed intruders ($\Gamma = 1$), Destroyed defenders ($\Gamma = 2$), Intercepted intruders ($\Gamma = 3$), Arrived intruders ($\Gamma = 4$). After four situations, some ASVs will be destroyed or arrives, and they are considered completed and subsequently removed from ACDS. A running flag, $\Xi_i = 1$, denotes that ASV _{i} is still operational, and $\Xi_i = 0$ signifies that ASV _{i} is done. In the first two situations, ASVs can be destroyed when they collide with ASVs of the same type, i.e., $\Xi_i = 0, \Xi_j = 0$, if $p_i^t = p_j^t, i \neq j, i, j \in \mathcal{V}_r$, or $i, j \in \mathcal{V}_d$. In the third situation, intruder i can be destroyed by defenders, i.e., $\Xi_i = 0, \Xi_j = 1$, once it is captured $p_i^t = p_j^t, i \in \mathcal{V}_r, j \in \mathcal{V}_d$. The fourth situation represents an intruder i arriving at region α , which is indicated by the arrival flag $\Omega_i = 1$ when $|p_i^t - p_\alpha| < r_\alpha$.

(4) Stop condition: an episode is completed when all the intruders are done ($\Xi_i = 0, \forall i \in \mathcal{V}_r$) or the allowable time T has elapsed, defined as $\Gamma = 5$. To proceed, we need to establish the following definition, as defenders are tasked with successfully protecting the region.

Definition 1 (Winning or Losing Condition of Defenders). Defenders $\mathcal{V}_d$ are considered to win (Win = 1), if and only if all the defenders $\mathcal{V}_d$ are alive ($\Xi_i = 1, \forall i \in \mathcal{V}_d$), and all the intruders $\mathcal{V}_r$ are intercepted before arriving at the destination ($\Xi_j = 0, \Omega_j = 0, \forall j \in \mathcal{V}_r$). Otherwise, defenders $\mathcal{V}_d$ are considered to lose (Win = 0).

2.3. A bi-level control strategy 双层的控制策略

As mentioned before, a bi-level controller is proposed to generate reference action data for defender ASVs. An ideal strategy for defenders is to pursue intruders while avoiding collisions. To achieve the strategy, we firstly introduce a set $\mathcal{N}_i^t$ to include the neighboring defenders of ASV _{i} ($i \in \mathcal{V}_d$), where $\mathcal{N}_i^t = \{j | \|p_{i,j}^t\| \leq d, j \in \mathcal{V}_d, j \neq i\}$, and d is a set value. 邻居集合

The control objective for a defender ASV _{i} can be divided into two situation: the first one is to pursue intruders when there is no neighbors, i.e.,

$$\lim_{t \rightarrow \infty} \|p_{i,j}^t\| = 0, j \in \mathcal{V}_r, \text{ if } \mathcal{N}_i^t = \emptyset. \quad (3)$$

Another situation is to avoid collisions when there exists neighbors, i.e.,

$$\|p_{i,j}^t\| - d \geq 0, \text{ if } \mathcal{N}_i^t \neq \emptyset, j \in \mathcal{N}_i^t. \quad (4)$$

Considering the underactuated nature of ASVs, we present a bi-level control framework to generate actions τ_i for achieving the control goals. The upper level is a position controller, and the lower level is an attitude controller. Noting that v_i cannot be controlled, the upper level outputs desired signals $[u_i^d, \omega_i^d]$ for $[u_i, \omega_i]$. Meanwhile, v_i is obtained by Eq. (2). As a result, the position p_i can be driven by Eq. (1) using $[u_i^d, v_i, \omega_i^d]$ to satisfy Eqs. (3), (4). The lower level outputs actions τ_i , allowing the real values $[u_i, v_i, \omega_i]$ to approach the desired values $[u_i^d, v_i^d, \omega_i^d]$ by Eq. (2).

2.3.1. The upper level control laws 上层控制律

For developing $[u_i^d, \omega_i^d]$ control laws addressing (3), we define the following error variables

$$z_e = \|p_{i,j}\|, \theta_e = \theta_d - \theta_i, \quad (5)$$

where θ_d is a defender ASV _{i} azimuth angle relative to a target intruder j , and $\theta_d = \arctan(y_{i,j}/x_{i,j})$. According to Eqs. (1), (2), (5), we have the following error dynamics

$$\begin{aligned}\dot{z}_e &= \cos(\theta_d) \dot{x}_j + \sin(\theta_d) \dot{y}_j - u_i \cos \theta_e - v_i \sin \theta_e, \\ \dot{\theta}_e &= \dot{\theta}_d - \omega_i.\end{aligned}\quad (6)$$

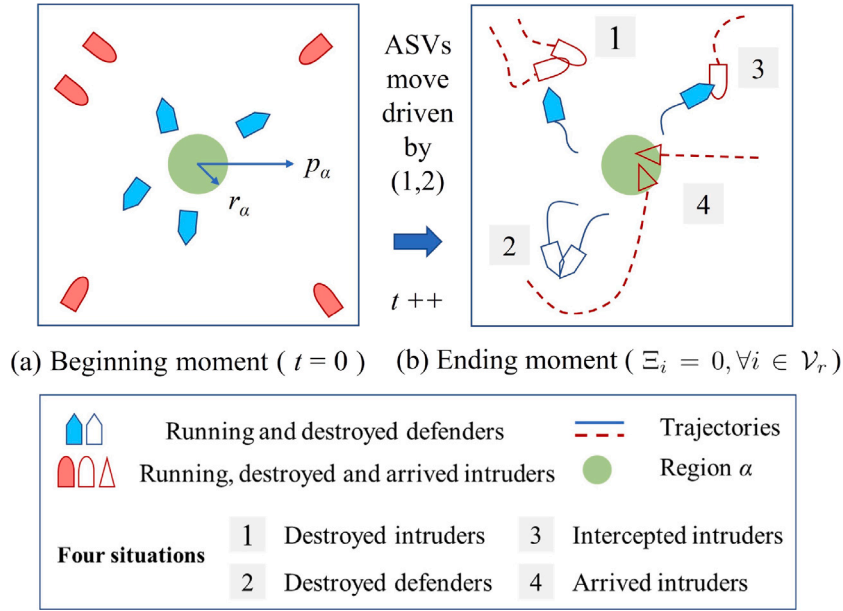


Fig. 2. The beginning and ending moments of ACDS. In the ending moment, four distinct situations occur ($\Gamma \neq \emptyset$), including destroyed intruders, destroyed defenders, intercepted intruders, and arrived intruders.

Then, the control laws of $[u_i^d, \omega_i^d]$ can be derived as

$$u_i^d = (\cos \theta_e)^{-1} \left[\cos(\theta_d) \dot{x}_j + \sin(\theta_d) \dot{y}_j - v_i \sin \theta_e + k_{z_e} (z_e - \delta_{xy}) \right], \quad (7)$$

$$\omega_i^d = k_{\theta_e} \theta_e + \dot{\theta}_d,$$

where k_{z_e} and k_{θ_e} are set control parameters, and δ_{xy} is a small positive value. When the condition $k_{z_e}, k_{\theta_e} \in \mathbb{R}^+$ is satisfied, the error variables z_e, θ_e can converge to zero, i.e., $\lim_{t \rightarrow \infty} z_e = 0, \lim_{t \rightarrow \infty} \theta_e = 0$.

To solve Eq. (4), we apply the artificial potential function [45]. Once $\mathcal{N}_i^t \neq \emptyset$, ASV_i will receive repulsion forces from the neighbors, so that ASV_i slow down and deviate or even stop to avoid collision.

2.3.2. The lower level control laws 下层控制律

For the lower-level controller, we employ an in-house code [46,47]. The control input $\tau_i = [\tau_i^u, \tau_i^\omega]^T$ is given as

$$\tau_i^u = -(k_1/k_3)u_i - (k_2/k_3)v_i\omega_i - (k_2/k_3)\tilde{u}_i - (k_1/k_3) \int \tilde{u}_i(s)ds, \quad (8)$$

$$\tau_i^\omega = \frac{k_4 + k_3 + k_4}{k_5}\omega_i + \frac{k_3^2 - (k_3 + k_4)k_3}{k_5}\tilde{\omega}_i.$$

If control parameters satisfy $k_1 > (1/4)k_2^2$ and $k_3, k_4 > 0$, then the actual velocity $[u_i, \omega_i]$ of i th vessel can be regulated to the upper level desirable velocity $[u_i^d, \omega_i^d]$, i.e.,

$$\lim_{t \rightarrow \infty} \tilde{u}_i = \lim_{t \rightarrow \infty} u_i - u_i^d = 0, \quad (9)$$

$$\lim_{t \rightarrow \infty} \tilde{\omega}_i = \lim_{t \rightarrow \infty} \omega_i - \omega_i^d = 0,$$

with $k_1 = -\frac{d_{1,j}}{m_{1,j}}, k_2 = \frac{m_{2,j}}{m_{1,j}}, k_3 = \frac{1}{m_{1,j}}, k_4 = -\frac{d_{3,j}}{m_{3,j}}, k_5 = \frac{1}{m_{3,j}}$. Finally, the bi-level controller of defenders is developed according to Eqs. (7), (8). The controller enables defenders to sample suitable actions τ_i , facilitating effective pursuit and interception of intruders without collisions.

入侵者双层控制器的改进

2.3.3. The bi-level controller modifications for intruders

For training defenders, most related research uses intruders with straight or random movements. However, these intruder strategies are highly inefficient, which may lead to the defenders not needing to take any actions to successfully protect the region. In such cases, training becomes unnecessary and fails to motivate policy updates. Therefore, designing an intelligent opponent is equally crucial. An ideal strategy

for intruders involves advancing towards the region α , while avoiding interception and collisions. With some modifications, the bi-level strategy is also adapted for intruders to achieve intelligent intrusion.

Firstly, in addition to collision avoidance, intruders also need to evade the interception of defenders. Thus, the neighboring defenders and intruders are all included in the neighbor set $\mathcal{N}_i^t$, i.e., $\mathcal{N}_i^t = \{j | \|p_{i,j}^t\| \leq d, j \in \mathcal{V}_d \cup \mathcal{V}_r, j \neq i\}$. Secondly, intruder i aims to approach the region α , which is a fixed area without moving velocities. Thus, the first equation in Eq. (6) can be simplified as

$$\dot{z}_e' = -u_i \cos \theta_e' - v_i \sin \theta_e', \quad (10)$$

where $z_e' = \|p_{i,\alpha}\|$, $\theta_e' = \theta_d' - \theta_i$, and $\theta_d' = \arctan(y_{i,\alpha}/x_{i,\alpha})$. Similarly, the first equation in Eq. (7) is modified as

$$u_i^d = (\cos \theta_e)^{-1} \left[-v_i \sin \theta_e + k_{z_e} (z_e - \delta_{xy}) \right], \quad (11)$$

Finally, all other control terms remain unchanged, and the modified bi-level control framework of intruders is constructed. The controller can provide qualified opponents for defenders to train better interception strategies.

2.4. Cooperative defense problem formulation 合作防御问题的描述

To model the cooperative defense process Markov decision processes are adopted and extended to a multi-agent system [48], which is defined by a set of quintuples (S, O_i, A_i, R_i) . The global state space S is an infinite set describing the possible properties of all defenders. O_i denotes the local observations of defender i , and each defender can receive the individual observation correlated with the state $S \mapsto O_i$. A_i represents the action space, which contains all actions of the agent. $S \times A_i \mapsto R_i$ is a reward function assigning an extrinsic reward r_i for taking an action a_i under state s_i .

Specifically, at each moment t ASV_i generates an action a_i^t by its policy π_i and the local observation $o_i^t, o_i^t \in s_i^t$, i.e.,

$$a_i^t = \pi_i(o_i^t). \quad (12)$$

As a result, ASV_i interacts with ACDS using a_i^t through Eqs. (1), (2), and gets to next state s_i^{t+1} with a reward r_i^t . This interaction process continues over time, generating a series of state transition quintuples $(s_i^t, a_i^t, r_i^t, s_i^{t+1})$ of ASV_i , which are stored into an experience replay buffer

D. Based on the samplings from the replay buffer D , the goal of each ASV is to optimize its policy $\pi_i(s_i^t)$ for generating an action a_i^t with the maximized long-term discounted cumulative reward R_i^t , i.e.,

$$R_i^t = r_i^{t+1} + \gamma r_i^{t+2} + \gamma^2 r_i^{t+3} + \dots = \sum_{k \in [0, T-t]} \gamma^k r_i^{t+k+1}, \quad (13)$$

where $\gamma \in [0, 1]$ denotes a discount factor. To obtain the optimum policy π_i^* , the state value function $\mathbb{E}_{\pi_i}(R_i^t | s_i)$ is adopted as the optimization goal, which is the expectation of the long-term discounted cumulative reward R_i^t of defender i with the state s_i .

Now, we are ready to develop the main technical problem for this paper.

Problem 1. For each defender governed by Eqs. (1), (2), how to design an algorithm to obtain the optimal action policy π_i^* , by maximizing $\mathbb{E}_{\pi_i}(R_i^t | s_i)$, i.e.,

$$\pi_i^* = \operatorname{argmax}_{a_i^t = \pi_i(o_i^t)} \mathbb{E}_{\pi_i}(R_i^t | s_i). \quad (14)$$

It is a notable fact that ACDS presents several challenges in addressing Problem 1. Firstly, The underactuated nature of ASVs introduces challenges to effective decision-making, as their actions act on nonintuitive and nonlinear dynamic systems. Secondly, defenders are responsible for safeguarding an entire area, whereas intruders only need to target a single point. This inherent asymmetry in the defense task significantly increases the difficulty for defenders. Additionally, intruders are more than defenders, coupled with the intruder avoidance abilities, which makes interception challenging. As a result, defenders have a limited time window for interception, requiring the development of efficient policies π_i to counter intruders. Lastly, defenders and intruders can be damaged during the region protection process, which adds a dynamic and nonstationary aspect to ACDS.

3. Cooperative defense method of ASVs ASV的合作防御方法

In this section, we introduce a cooperative defense method for ASVs, namely ACDM, to address Problem 1 within the specific challenges. Firstly, we employ an actor neural network $\pi_{\mu_i}(o_i^t)$ parameterized by μ_i , to approximate a defender policy π_i . This transforms Problem 1 into an optimization task of μ_i . Secondly, due to the adaption of neural networks, ACDM can leverage the benefits of two kinds of learning methods to tackle the challenges of Problem 1, including supervised learning and deep reinforcement learning. Supervised learning mitigates the issues of weak convergence and low sampling efficiency in reinforcement learning, while reinforcement learning pushes the upper performance limits of supervised learning. Initially, behavior cloning, a supervised learning technique, is used to learn the bi-level control strategy. This approach accelerates the convergence of μ_i to local optima. As the training progresses, the deep reinforcement learning method MADDPG plays a more prominent role, enabling defenders to explore more efficient policies to enhance performance. Finally, we carefully design elements for ACDM, particularly reward functions, to facilitate effective policy updates. Here are the details of ACDM.

3.1. Behavior cloning

For training actor networks $\pi_{\mu_i}(o_i^t)$, most learning-based methods are model-free algorithms, which adopt blind exploration. However, these methods frequently encounter issues with slow convergence or even non-convergence. Fortunately, leveraging expert data can significantly expedite the initial learning process. On the other hand, the expert data should preferably include the dynamics information to solve the underactuated nature. Thus, we employ the bi-level control strategy in Eqs. (7), (8) as expert data, which generates actions τ_i^t based on local observations o_i^t , promoting defenders to pursue intruders without collisions. This pair (o_i^t, τ_i^t) is labeled as the j th observation-action data

(o_k, τ_k) and stored in a dataset B , preparing for training $\pi_{\mu_i}(o_i^t)$. Simultaneously, the action networks $\pi_{\mu_i}(o_i^t)$ also choose actions a_k based on the corresponding observations o_k . Since the actions are continuous, the loss function for behavior cloning $\mathcal{L}_{BC}$ is defined as the mean square error of τ_i^t and a_i^t , represented as

$$\mathcal{L}_{BC} = \eta \lambda \mathbb{E}_{(o_k, \tau_k) \sim B} [(\tau_k - \pi_{\mu_i}(o_k))^2], \quad (15)$$

where η represents the influence strength of behavior cloning on policy updates, and λ is a decay coefficient from 1 to 0. As λ decreases, the actor networks $\pi_{\mu_i}(o_i^t)$ gradually transition from pure imitation, which enables ASVs to have autonomy once a specific level of cumulative reward is achieved, rather than consistently adopting behavior cloning. As λ approaches zero, defenders make decisions based only on its own exploration of reinforcement learning. This continuous update of the actor network allows defenders to strike a balance between imitation and exploration.

3.2. MADDPG

MADDPG is a reinforcement learning method that follows an actor-critic framework. Specifically, since the cumulative reward R_i^t in Eq. (13) is hard to solve, a critic neural network $Q_i^\pi(s, a)$ parameterized by ε_i is usually adopted to approach R_i^t for each defender. Meanwhile, actor network are similarly required to approximate ASV policies π_i . Hence, the actor networks $\pi_{\mu_i}(o_i^t)$ trained by behavior cloning are also adopted here, allowing for further optimization of $\pi_{\mu_i}(o_i^t)$. π_{μ_i} choose actions a_i^t according to the local observation o_i^t , whereas the critic network Q_i^π produce an approximate value $\hat{Q}_i$ of R_i^t , so as to evaluate the action decided by the actor network. Besides, critic networks require co-agent information, such as states s and actions a , to handle the nonstationarity of ACDS.

To solve Problem 1, the policy gradient $\nabla_{\mu_i} J(\mu_i)$ [3] is firstly derived as:

$$\nabla_{\mu_i} J(\mu_i) = \mathbb{E}[\nabla_{\mu_i} \pi_{\mu_i}(a_i | o_i) \nabla_{\mu_i} Q_i^\pi(s, a_{1, \dots, N} | a_i = \pi_{\mu_i}(o_i))]. \quad (16)$$

To approach the cumulative reward R_i^t , the goal of the critic network Q_i^π is to minimize the sum of square loss $L(\varepsilon_i)$, between the critic network output $\hat{Q}_i$ and R_i^t , i.e., $L(\varepsilon_i) = \min_{\varepsilon_i} \mathbb{E}[(\hat{Q}_i - R_i)^2]$. Thus, the gradient of critic networks $\nabla_{\varepsilon_i} L(\varepsilon_i)$ is calculated by

$$\nabla_{\varepsilon_i} L(\varepsilon_i) = \mathbb{E}[(Q_i^\pi(s, a_{1, \dots, N_i}) - y) \cdot \nabla_{\varepsilon_i} Q_i^\pi(s, a_{1, \dots, N_i})], \quad (17)$$

where $y = r_i^t + \gamma Q_i^{\pi'}(s^{t+1}, a_{1, \dots, N}^{t+1} | a_i^{t+1} = \pi_i'(o_i))$, which is an approximate value calculated by the temporal difference (TD), and $Q_i^{\pi'}$, π_i' represent the target networks with delayed parameters ε_i' , μ_i' . Sampling transitions $(s_i^t, a_i^t, r_i^t, s_i^{t+1})$ from the replay buffer D , the network parameters μ_i, ε_i are iteratively updated by the gradients $\nabla_{\mu_i} J(\mu_i), \nabla_{\varepsilon_i} L(\varepsilon_i)$. Additionally, MADDPG relies on gradient-based methods, which necessitate high-quality initial actor parameters μ_i for effectively training $\pi_{\mu_i}(o_i^t)$. This requirement is addressed through behavior cloning, where the quality of μ_i is initially improved by Eq. (15).

3.3. The framework of the proposed ACDM

The ACDM framework is represented in Fig. 3, which combines the strengths of behavior cloning and MADDPG. Behavior cloning is employed in the initial phase to expedite convergence towards local optima, while MADDPG explores more efficient policies to improve the defense performance. Operators representing ACDM, MADDPG, and behavior cloning are color-coded as grey, blue, and green, respectively. Numerical labels denote the sequence of execution for the operators. The red arrow lines indicate the direction of action transmission. The crux of ACDM is the grey operator 5, namely policy updates, where Problem 1 in Eq. (14) is reformulated into Eq. (18) by combining the two methods.

$$\pi_{\mu_i}^* = \operatorname{argmax}_{a_i^t = \pi_{\mu_i}(o_i^t)} [\mathbb{E}_{\pi_i}(\hat{Q}_i | s_i) - \mathcal{L}_{BC}], \quad (18)$$

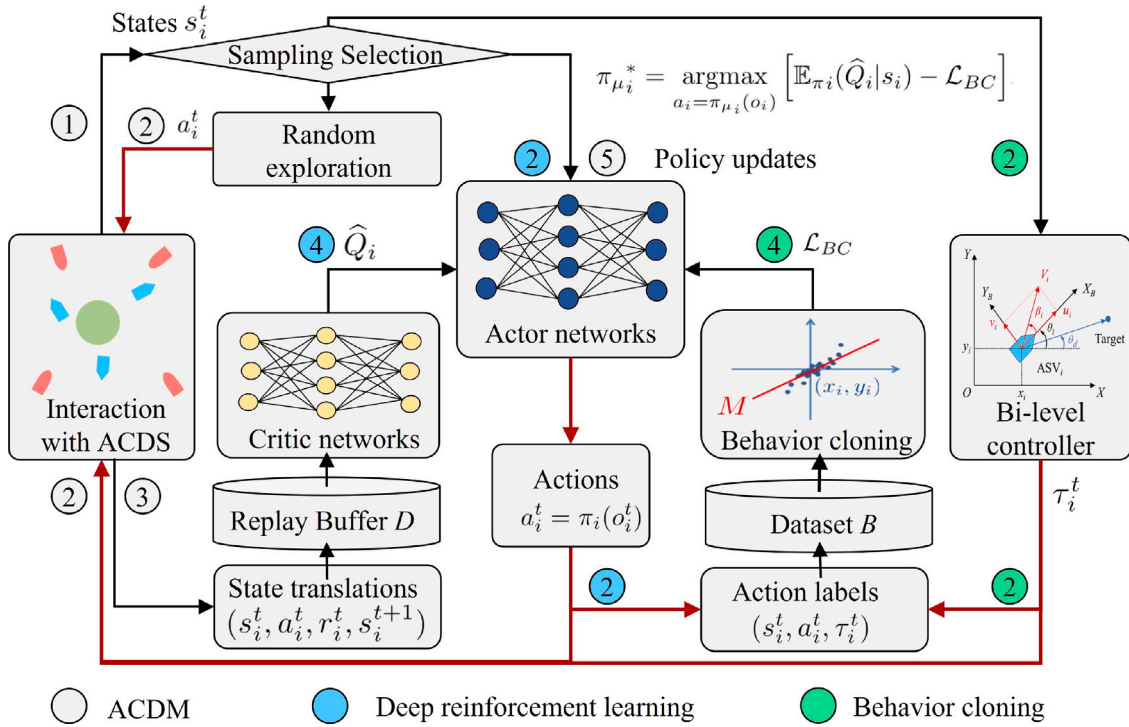


Fig. 3. ACDM framework adopts behavior cloning for convergence acceleration, and employs MADDPG for policy exploration. In the diagram, ACDM-related operators are denoted in grey, MADDPG in blue, and behavior cloning in green. Numerical labels signify the execution sequence of these operators within the framework.

where $\mathbb{E}_{\pi_i}(\hat{Q}_i | s_i)$ denotes the predicted cumulative rewards calculated by critic networks Q_i^{π} , and the loss function of behavior cloning $\mathcal{L}_{BC}$ is obtained from Eq. (15). $\mathcal{L}_{BC}$ gradually decreases to 0 as the training progresses, making Eqs. (14) and (18) equivalent. Policy parameters are updated by $\mu_i^{k+1} = \mu_i^k + \epsilon(\nabla_{\mu_i} J(\mu_i) - \nabla \mathcal{L}_{BC})$, where ϵ is the learning rate.

All operators are executed after an interaction process, during which ASV actions a_i^t affect ACDS and induce changes in their states s_i^t . Now, an overview of all the operators are provided as follows.

Initially, ACDS provides the states s_i^t of a moment t for all defenders (grey operator 1). Thereafter, a selection procedure chooses an action generator among three options: random explorations (grey 2), actor networks (blue 2), and bi-level controllers (green 2). In the early training stages, bi-level controllers are predominantly selected to generate actions τ_i^t , while the probability of selecting actor networks and random exploration increases as training progresses.

For the grey 2, Random exploration generates actions using the following formulas: $\tau_i^u = \varpi \cdot \tau_{\max}^u$, $\tau_i^o = -\tau_{\max}^o + 2 \cdot \varpi \cdot \tau_{\max}^o$ (grey 2), where $\varpi \in [0, 1]$ is a uniformly distributed value. These actions are then used to interact with ACDS, generating the next states s_i^{t+1} (grey 3).

For the green 2, the controller actions τ_i^t are converted into labels for behavior cloning (green 2 and 4), so that the action loss $\mathcal{L}_{BC}$ is calculated using Eq. (15) for policy updates. Meanwhile, τ_i^t is also used in interactions with ACDS for obtaining the next states s_i^{t+1} (grey 2).

Similarly, for the blue 2, actions generated by actor networks a_i^t are used to compute $\mathcal{L}_{BC}$ (blue 2) and interact with ACDS (grey 2). Subsequently, using state translations $(s_i^t, a_i^t, r_i^t, s_i^{t+1})$ stored in the replay-buffer D , $\hat{Q}_i$ are calculated through the critic networks Q_i^{π} (blue 4).

Finally, the core operation (grey 5) in Eq. (18) is executed for defender training, addressing Problem 1 through consistent interactions.

3.4. The proposed elements in ACDM

It is crucial to design ACDM elements carefully for guaranteeing the effectiveness. In addition to the previously defined action space τ_i , these

elements include the state s_i^t and the reward function r_i^t . The specific techniques are elaborated below.

State space with two moment properties: Research has revealed that the incorporation of sequential states significantly contributes to decision-making. Consequently, the state space s_i is defined based on the properties χ_i at the current and the previous moments, represented as $s_i^t = [\chi_i^{t-1T}, \chi_i^{tT}]^T$. Each moment property χ_t includes information about defender i itself, as well as its relative situations with other ASVs and intruders. Self-information encompasses position p_i , surge velocity u_i , sway velocity v_i , heading angle θ_i , yaw velocity ω_i , and the running flag Ξ_i . For the relative situations, accessing precise data of other ASVs or intruders can be challenging. Hence, only the relative positions $p_{i,j}$ are adopted, making it suitable for practical scenarios. Lastly, the property is expressed as

$$\chi_i^t = [p_i^T, u_i, v_i, \theta_i, \omega_i, \Xi_i, p_{i,j}^T]^T, j \in \mathcal{V}_d \cup \mathcal{V}_r, j \neq i. \quad (19)$$

Reward functions of hybrid terms: Real-time rewards needs to accurately approximate team defense performance, and we design an innovative reward function with hybrid terms, expressed as

$$r_i^t = r_{\text{sit}_i}^t + r_{\text{der}_i}^t + r_{\text{mod}_i}^t, \quad (20)$$

where $r_{\text{sit}_i}^t, r_{\text{der}_i}^t, r_{\text{mod}_i}^t$ denote the situation, the derivative and the modification rewards, respectively. $r_{\text{sit}_i}^t$ is designed based on defense situations, such as $p_{i,j}^t, \theta_d$ in Fig. 1. It encourages defenders to pursue intruders effectively, while $r_{\text{der}_i}^t$ further enhances this promoting effect. However, $r_{\text{sit}_i}^t$ and $r_{\text{der}_i}^t$ do not contain the influence of interception results and the four situations in Fig. 2(b). Hence, $r_{\text{mod}_i}^t$ is introduced to modify defender behavior according to the results or situations. All the terms cooperatively shape the reward R_i to align with task objectives, so that the actor network parameters μ_i can be prevented from falling into local optima.

The design of $r_{\text{sit}_i}^t$ follows a similar approach to the mainstream method, incorporating four terms. Firstly, to address the scenario where intruders outnumber defenders, we designate the nearest intruder of defender i as the target $\tilde{i}$, and $\tilde{i}$ is related to the time. To discourage defenders from switching constantly among different intruders, the

first two terms are designed with the target $\tilde{i}$, which all multiply attention weights k_1, k_2 to amplify the impact of $\tilde{i}$. The first term is a distance reward, represented as $k_1(\|\rho_{i,\alpha}^t\| - \|\rho_{i,\tilde{i}}^t\|)$, which contains a positive intruder-region distance for urging defenders to keep the target intruder out from the region α . Besides, it also includes a negative defender-intruder distance, motivating defenders to approach their nearest intruders. The second term $k_2 \cos(|\theta_i - \theta_d|)$ regulates the defender's heading angle towards the target intruder. The significance of heading accuracy is underscored by the fact that deviations from the target intruder result in progressively larger distances over time. Hence, the coefficients are set with $k_2 > k_1$ to preferentially minimize heading errors. The third term $\sum_{j \in \mathcal{V}_r} \|\rho_{j,\alpha}^t\|/n$ promotes collaborative behavior among defenders to drive intruders out. Although it does not directly affect the actions of individual defenders towards specific intruders, it encourages smoother trajectories between interceptions of different target intruders. The last term $\sum_{j \in \mathcal{N}_i^t} (1 - \|\rho_{i,j}^t\|/d)^2$ considers defender-defender distances to ensure collision avoidance and the safe navigation of defenders. Finally, $r_{\text{sit}_i}^t$ is represented as:

$$r_{\text{sit}_i}^t = k_1(\|\rho_{i,\alpha}^t\| - \|\rho_{i,\tilde{i}}^t\|) + k_2 \cos(|\theta_i - \theta_d|) + \sum_{j \in \mathcal{V}_r} \|\rho_{j,\alpha}^t\|/n - \sum_{j \in \mathcal{N}_i^t} (1 - \|\rho_{i,j}^t\|/d)^2. \quad (21)$$

By considering states with two moment properties $s_i^t = [\chi_i^{t-1\top}, \chi_i^{t\top}]^\top$, the derivative reward $r_{\text{der}_i}^t$ can be obtained. This reward captures the change in $r_{\text{sit}_i}^t$, which encourages ASVs to make progressive moves while punishing retrogressive ones. $r_{\text{der}_i}^t$ helps to compress the final reward, expressed as:

$$r_{\text{der}_i}^t = r_{\text{sit}_i}^t - r_{\text{sit}_i}^{t-1}. \quad (22)$$

Continuous rotations of ASVs to adjust their heading angles are discouraged. To reduce the influence of heading angles, the coefficients k_1, k_2 of $r_{\text{sit}_i}^t, r_{\text{sit}_i}^{t-1}$ in Eq. (22) are set as $k_1 > k_2$.

However, there are specific situations that cannot be sufficiently captured by $r_{\text{sit}_i}^t$ and $r_{\text{der}_i}^t$, such as ASV damage or protection success. In such cases, the reward r_i^t is typically adjusted directly by adding a sharp value S_v , which aims to accommodate sudden changes but results in high volatility for learning. To alleviate the impact of these abrupt changes, we introduce a modification reward $r_{\text{mod}_i}^t$, which decomposes the sharp value S_v into an arithmetic progression Λ .

Firstly, the sharp value S_v is carefully defined according to the typical situations: (a) Destroyed intruders ($\Gamma = 1$): $S_v = 0.5 \cdot T_e$, if the target intruder $\tilde{i}$ collides. $S_v = 0.5 \cdot T_e/n$, if other intruders collides. (b) Destroyed defenders ($\Gamma = 2$): $S_v = -2T_e$, if the defender is damaged. (c) Intercepted intruders ($\Gamma = 3$): $S_v = 2T_e$, if the intruder $\tilde{i}$ is intercepted. (d) Arrived intruders ($\Gamma = 4$): $S_v = -T_e$, if the target intruder $\tilde{i}$ arrives. $S_v = -T_e/n$, if other intruders arrives. (e) End of defense ($\Gamma = 5$): $S_v = -5T_e$, if Win = 1, and $S_v = 5T_e$, otherwise.

Secondly, since the occurring situations are more influenced by previous nearby actions, the rewards should correspondingly be given bigger absolute values. Thus, for the arithmetic progression Λ , the closer sequences are to the situation, the larger absolute values are assigned. Specifically, the initial term of Λ is set as 0, and the common difference of Λ is represented as d_Λ . When a situation arises on the moment T_e ($\Gamma \neq \emptyset$), the term number n_Λ of Λ is calculated by $n_\Lambda = T_e/\Delta t + 1$, where Δt is the time interval. Thus, the sum Sum_Λ of Λ is obtained by

$$\text{Sum}_\Lambda = \frac{n_\Lambda(n_\Lambda - 1)d_\Lambda}{2} = \frac{(T_e + \Delta t)T_e}{2\Delta t^2} d_\Lambda. \quad (23)$$

Thirdly, to commensurately decompose S_v , we use the equation $\text{Sum}_\Lambda = S_v$. The common difference is then obtained as $d_\Lambda = S_v \cdot \frac{2\Delta t^2}{T_e(T_e + \Delta t)}$.

Finally, the modification reward $r_{\text{mod}_i}^t$ corresponds to the $(t/\Delta t + 1)$ th term of Λ , represented as

$$r_{\text{mod}_i}^t = S_v \cdot \frac{2\Delta t}{T_e(T_e + \Delta t)} \cdot t, \quad (24)$$

In conclusion, a complete calculation process of rewards r_i^t is summarized in Algorithm 1.

Algorithm 1 Interaction with ACDS

- 1: Input actions of $a = (a_1, \dots, a_{m+n})$ all ASVs, and drive ASV moves by Eqs. (1), (2).
- 2: Obtain the global state S_{t+1} after a time interval Δt .
- 3: Receive the situation and the derivative rewards $r_{\text{sit}_i}^t, r_{\text{der}_i}^t$ by Eqs. (21), (22).
- 4: if the four situations in Fig. 2 occur, i.e., $\Gamma \neq \emptyset$, then
- 5: Calculate rewards $r_{\text{mod}_i}^t$ using Eq. (24), and add $r_{\text{mod}_i}^t$ to every r_i in the buffer D of this episode.
- 6: Break if ACDS is end, i.e., $\Gamma = 5$.
- 7: end if
- 8: Output and store the transition (S_t, a, r, S_{t+1}) in replay buffer D

Algorithm 2 outlines the proposed ACDM, encompassing steps 1–26 for the training process, with step 27 dedicated to the autonomous decision-making process in ACDS. The reward functions serve as the unique criteria in optimizing actor parameters μ_i , and their design is meticulously tailored to the objective and outcomes of a defender team. Consequently, these functions enable the precise regulation of each agent's behavior to collectively accomplish a shared task after training.

Algorithm 2 ACDM

- 1: Initialize actor and critic networks π_μ, Q_i^π , a random process σ for action exploration, the influence strength of behavior cloning $\eta = 1$.
- 2: for episode = 1 : the max episodes N do
- 3: for $t = 1 : \Delta t : T_{\text{tot}}$ do
- 4: Receive states $s_i^t = [\chi_i^{t-1\top}, \chi_i^{t\top}]^\top$ with two moment properties.
- 5: if $\varpi < \eta$ then
- 6: // Behavior cloning
- 7: Generate actions τ_k according to s_i^t through the bi-level controllers in Eqs. (8)–(11).
- 8: Store τ_k and $a_k = \pi_{\mu_i}(o_i^t)$ in the dataset B , and calculate the actor loss $\mathcal{L}_{BC}$ by Eq. (15).
- 9: Call Algorithm 1 to interact with ACDS using τ_k , for supplying replay-buffer D .
- 10: else
- 11: // MADDPG
- 12: For each defender i , select actions $a_i^t = \pi_i(o_i^t) + \sigma_i$.
- 13: Call Algorithm 1 using a_i^t to supply replay-buffer D .
- 14: for defender $i = 1 : m$ do
- 15: Sample a random minibatch of J_s samples (S_t, a, r, S_{t+1}) from D
- 16: Set $y = r_i + \gamma Q_i^\pi(s^t, a_{i', \dots, N}^t | a_{i'} = \pi_{i'}(o_{i'}))$
- 17: Compute critic gradient estimation to minimize the loss by Eq. (17):
 $\nabla_{\epsilon_i} L(\epsilon_i) = \mathbb{E}[(Q_i^\pi(s, a_{1, \dots, N}) - y) \cdot \nabla_{\epsilon_i} Q_i^\pi(s, a_{1, \dots, N})]$
- 18: Compute actor gradient estimation by Eq. (16):
 $\nabla_{\mu_i} J(\mu_i) = \mathbb{E}[\nabla_{\mu_i} \pi_{\mu_i}(a_i | o_i) \nabla_{\pi_{\mu_i}} Q_i^\pi(s, a_{1, \dots, N} | a_i = \pi_{\mu_i}(o_i))]$
- 19: Update target critic networks for each defender i :
 $\epsilon_i^{k+1} = \epsilon_i^k - \alpha_c \nabla L(\epsilon_i), \epsilon_i^t \leftarrow \tau \epsilon_i + (1 - \tau) \epsilon_i^t$
- 20: end for
- 21: end if
- 22: Combine policy gradients of the behavior cloning $\nabla \mathcal{L}_{BC}$ and MADDPG $\nabla_{\mu_i} J(\mu_i)$.
- 23: Update policies by $\mu_i^{k+1} = \mu_i^k + \epsilon (\nabla_{\mu_i} J(\mu_i) - \nabla \mathcal{L}_{BC})$ to address the variant of Problem 1 in Eq. (14).
- 24: end for
- 25: end for
- 26: Obtain the optimum μ^* and the corresponding actor network $\pi_{\mu_i}^*$
- 27: Autonomously decide protection actions by $\pi_{\mu_i}^*$ in ACDS, i.e., $a_i^t = \pi_{\mu_i}^*(o_i^t)$

Table 1
ACDS configuration.

Category	Defenders $\mathcal{V}_d$	Intruders $\mathcal{V}_r$	Dense area α	ACDS
Number (2 cases)	$m = 2, m = 3$	$n = 3, n = 5$	Position $p_\alpha = [250, 250]$	Edge $b_s = 500$
Mobility $\tau_{\max}^\mu, \tau_{\max}^\omega$	500, 300	300, 180	Radius $r_\alpha = 30$	Total time $T = 60$ s

Table 2
Structure and parameter of actor and critic networks.

Networks	Activation function and neurons			Learning rates
	Input layer in two cases	3 hidden layers	Output layer	
Actor	ReLu, 38 and 56	ReLu, 64	SoftMax, 2	Optimizer
Critic	ReLu, 80 and 204	ReLu, 64	None, 1	Adam

4. Numerical simulations

To assess the effectiveness of the proposed ACDM, numerical simulations are performed in two distinct ACDS cases. The simulations involve a comparative analysis of ACDM against pure behavior cloning and MADDPG, along with an exploration of variations in reward structures.

4.1. Parameter settings of scenarios and networks

The configuration of scenario parameters and network settings plays a crucial role in the training process. Scenario parameters should be chosen based on the specific interception requirements, including the location and size of the protected region. For ACDS, the scenario parameters are detailed in Table 1, and the ASV model data is based on the vessel HUSTER-16 [49]. Performance evaluation involves testing each method in two different ACDS cases: one with 2 defenders versus 3 intruders, and the other with 3 defenders versus 5 intruders. In these cases, ACDS operates within a coordinate system ranging from [0, 500] in both dimensions. The defense region α is centered at $p_\alpha = (250, 250)$ with a radius of $r_\alpha = 30$. Given that there are more intruders than defenders, it is challenging for the defenders to sequentially intercept all attackers without a mobility advantage. Consequently, the maximum input values for defenders are set to $\tau_{\max}^\mu = 500$ and $\tau_{\max}^\omega = 300$, while the corresponding values for intruders are 300 and 180. The limited time T is set to 60 s, which is sufficient for all intruders to reach their destination without interference by defenders. Importantly, all parameters are normalized during the calculation of agent states to enhance learning performance.

The selection of MADDPG parameters can be guided by the recommendations in [50]. Based on prior experiences, we set the discount factor γ to 0.95, and the learning rates ϵ for both the actor and critic networks are set to 0.02. The replay buffer size is 5×10^5 , and the batch size of updating is 256. For region protections, networks with 5 dense layers have demonstrated favorable performance. For the actor networks, the inputs consist of the local observations $o_i^t = s_i^t$, and the outputs are actions a_i^t , represented as $[\tau_i^\mu, \tau_i^\omega]$. In the critic networks, the inputs include all states $S_t = (s_1^t, s_2^t, \dots, s_m^t)$ and actions $a = (a_1^t, a_2^t, \dots, a_m^t)$, while the output is an approximate value $\hat{Q}_i$ for R_i^t . The dimensions of inputs and outputs are determined accordingly, and the architecture of the actor and critic networks can be provided in Table 2, including the layers and the corresponding activation functions.

4.2. Evaluation criteria of cooperative defense performance

For the evaluation criteria, three metrics are defined, including success rates (SRs), performance index P_I , and team reward function R_T . SRs serve as the most fundamental criterion for assessing the methods. To mitigate the impact of randomness, defenders trained by each method autonomously execute the mission in 1×10^5 different episodes for each case, and the number of successful protections is determined according to Definition 1.

Beyond SRs, a comprehensive set of more detailed metrics are chosen for data collection and in-depth analysis. These metrics encompass the number of intercepted intruders n_I , the count of surviving defenders m_s , and the completion time T_e . To differentiate the influence of these metrics, each of them is assigned different weights. Thus, the performance index P_I is expressed as

$$P_I = \begin{cases} n_I \cdot 10^4 + m_s \cdot 10^3 + (T - T_e), & \text{if Win} = 1, \\ n_I \cdot 10^4 + m_s \cdot 10^3 + T_e, & \text{if Win} = 0, \end{cases} \quad (25)$$

For instances where the mission is successfully completed (Win = 1), a shorter completion time $(T - T_e)$ is preferred, as it reflects more effective defense. Conversely, in cases where the mission is not successful (Win = 0), a longer completion time T_e is favored, indicating that the defenders exerted maximum effort to delay the intruders. In brief, P_I allows for a thorough assessment of the methods performance by considering multiple aspects of the defense mission.

Subsequently, to examine the learning effect, we introduce the team reward R_T , which is the accumulation of the true rewards of all defenders in one episode, calculated as follows:

$$R_T = \sum_{i \in T_e} \sum_{i \in \mathcal{V}_d} r_i^t \quad (26)$$

Finally, it is expected that these three criteria will exhibit similar growing trends as methods become more effective in training.

4.3. Validation of the bi-level control strategy

The bi-level controller plays a crucial role in the effectiveness of ACDM. There are two main aspects to its significance: Firstly, the bi-level controller allows for the efficient movement of intruders. It guides them to swiftly approach the protected area while avoiding interception and collisions, which is more competent than random actions used in many learning research scenarios, providing a foundation for training intelligent defenders. Secondly, this controller provides reference behaviors for defenders to imitate, significantly influencing the learning process and results of ACDM.

To demonstrate the effectiveness of the bi-level control strategy, simulations are conducted to compare it with a random controller. Thus, defenders execute stationary or inefficient protection movements. The inefficient defenders are controlled by the bi-level controller but with lower inputs $\tau_i^{\max}$. The results are depicted in Fig. 4. In a series of ten simulations, the random intruders consistently fail to reach the region. In contrast, all intruders employing the proposed strategy safely arrived within less than 52 s, when confronted by static or inefficient defenders. Specifically, in Fig. 4(a), random intruders face static defenders, and they could not reach the region despite the absence of defender disturbances. In Fig. 4(b), random intruders confront inefficient defenders. Intruders 4 and 5 remain almost stationary, and only exhibit irregular rotations. Intruders 6–8 take some actions, but still cannot progress effectively towards the protected region. Finally, all intruders are intercepted, validating the effectiveness of the defenders driven by the bi-level controller. In contrast, Fig. 4(c-d) depict the outcomes of the proposed intruder strategy, where all intruders

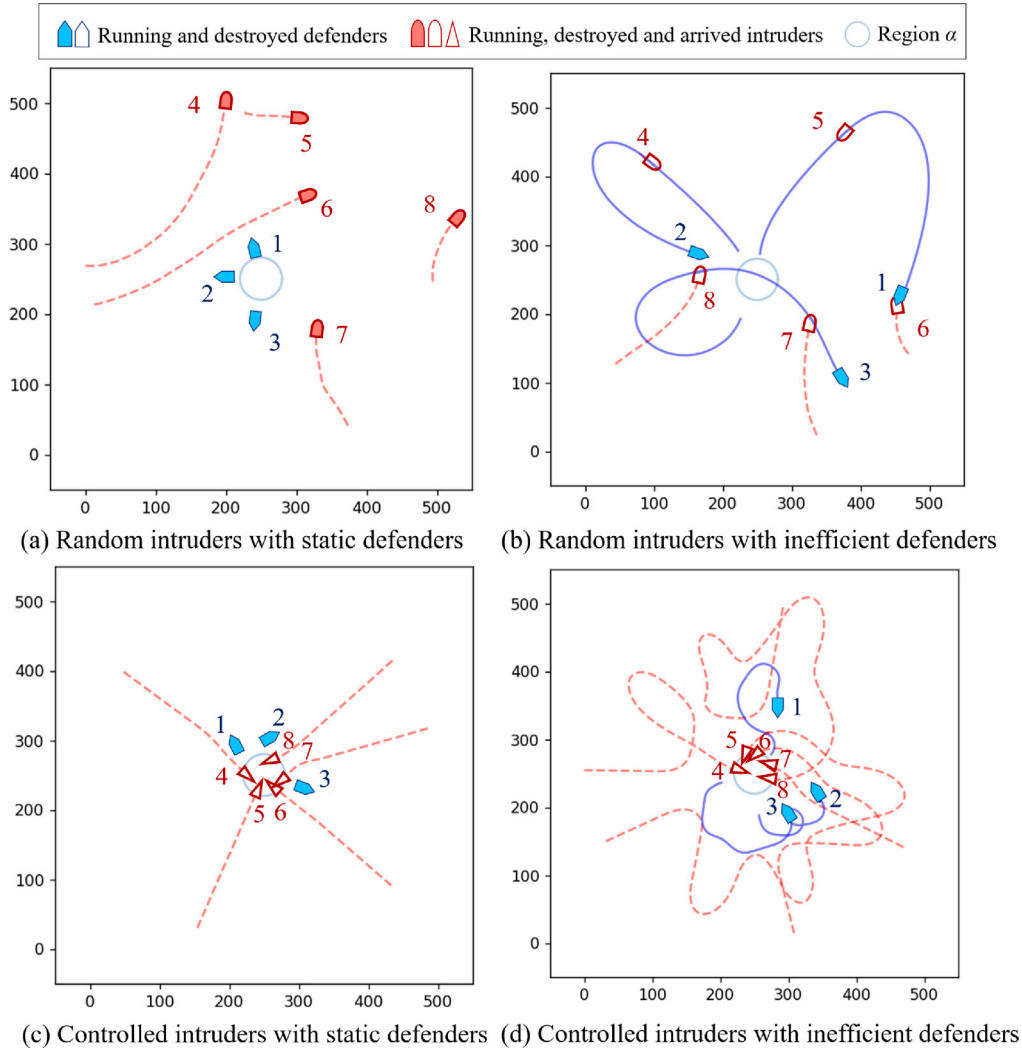


Fig. 4. Four examples of intruders controlled by the random and the proposed strategy.

successfully reach the protected region. In Fig. 4(c), intruders 4 and 7 take actions to avoid collisions, demonstrating their adaptive and intelligent behavior. On the other hand, the remaining intruders moved directly towards the protected region without additional evasive movements, as the defenders are not positioned in their heading directions. For Fig. 4(d), all intruders exhibited excellent obstacle avoidance and intrusion performance. These examples underscore the superiority of the bi-level controller.

4.4. Comparison between different methods

Given that ACDM is a composite approach that integrates both behavior cloning and MADDPG, these three distinct methods are employed for the numerical simulations and comparative evaluations. It is crucial to underscore that the outcomes of behavior cloning serve as a benchmark, illustrating the performance of supervised learning and conventional control techniques. Besides, MADDPG is recognized as a well-established reinforcement learning algorithm, widely acknowledged for its efficacy in multi-agent protection and confrontation scenarios [30,42–44,50]. These methodological choices provide a robust foundation for the systematic assessment and comparison of ACDM performance.

Table 3 presents SRs achieved by these methods over 100,000 episodes for both ACDS cases. The results indicate that ACDM outperforms the other methods, achieving the highest SRs of 91.68% in case

1 and 80.38% in case 2. In contrast, behavior cloning achieves SRs of 89.76% and 31.47% in the two cases, while the MADDPG records SRs of 9.84% and 2.73%. As the number of ASVs increases from case 1 to case 2, there is a decrease in SRs for all methods. Notably, behavior cloning experiences the most significant decrease, revealing its limitations in complex scenarios. This can be attributed to its reliance on control laws with fixed rules, which leads to poor adaptability. Besides, the importance of reinforcement learning is proven, as it can continuously explore unknown policies to improve decision-making performance. On the other hand, SRs of MADDPG are the lowest, indicating that the method may get stuck in local optima. Thus, it is important to recognize that behavior cloning can provide valuable starting points for MADDPG, enhancing learning efficiency in the ACDM framework. This combination of methods leverages the strengths of supervised learning, traditional control, and reinforcement learning, contributing to ACDM's superior performance.

To monitor the training process, the rewards R_T and performances P_I curves for these methods in the two cases are shown in Figs. 5–6. The methods are tested every 10 training episodes, and the average results of 10 experiments are reported. According to the learning curves, all methods exhibit a consistent upward in both rewards R_T and performances P_I , validating the effectiveness of training processes. Besides, these curves follow similar trends across all methods, underscoring the correctness of the reward design. Since the intercepted intruder number cannot be reflected by reward curves, it is necessary to use the

Table 3
SRs of the methods and the rewards in 1×10^5 episodes for the two ACDS cases.

Category	The proposed	Comparison methods		Reward variations		
Method	ACDM	Cloning	MADDPG	R-s	R-s-d	R-s-m
SRs of Case 1 (%)	91.68	89.76	9.84	90.63	91.20	91.52
SRs of Case 2 (%)	80.38	31.47	2.73	22.51	54.36	71.74

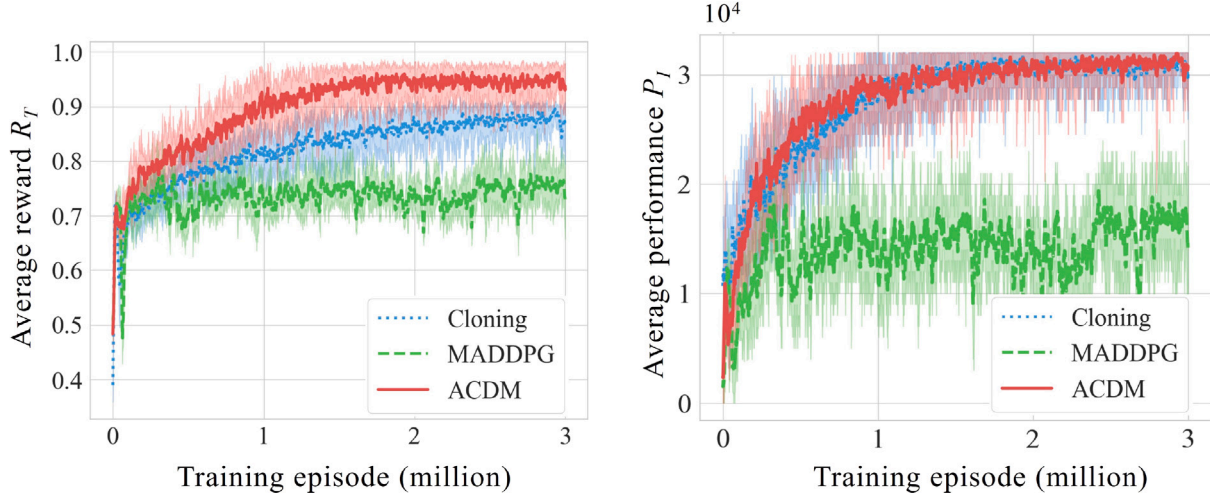


Fig. 5. Learning curves of the different methods for case 1. The left graph displays the average reward for each generation, while the right one shows the average fitness. Each method is distinguished by a different color. The shaded areas represent the confidence intervals for each method, indicating the mean $\pm$ standard deviation across ten different random episodes.

performance curves as evaluation criteria. According to P_I , defenders are still able to intercept some intruders and survive in most failure protections. This demonstrates the difficulty of achieving successful protection. Among these methods, ACDM stands out by achieving the highest level curves, reflecting its superior performance. Behavior cloning exhibits the most consistent line shape between rewards and performances, proving that the controller can generate reasonable actions for ASVs to effectively learn. However, when examining the P_I curves, it becomes evident that the controller has inherent performance limitations, primarily capable of intercepting 3–4 intruders in ACDS scenarios. Consequently, in the case of scenario 1, the performance difference between ACDM and behavior cloning is relatively insignificant, but in the more complex scenario 2, this difference becomes notably pronounced. This contrast emphasizes the necessity of incorporating MADDPG into the ACDM framework. Besides, MADDPG often gets trapped in local optima, resulting in learning curves consistently lingering at lower performance levels. Meanwhile, these curves exhibit more significant fluctuations. Notably, in scenario 2, MADDPG can intercept more intruders than in scenario 1, highlighting the exploration capability inherent to reinforcement learning, which is not affected by more complicated scenes. This suggests that by providing a solid starting point, MADDPG can potentially be further improved.

4.5. Exploration of variations in reward structures

In this section, we examine the relationship between different rewards and defense performance. Specifically, different terms in Eq. (20) are removed respectively to assess their individual impacts. It is worth noting that the situation reward $r_{sit_i}^t$ is the fundamental term. Thus, four types of reward variations are tested in ACDM frameworks, including R-s, R-s-d, R-s-m, and the proposed reward r_i^t in Eq. (20). The method using the proposed r_i^t is actually ACDM. R-s is the mainstream reward design method. It uses $r_{sit_i}^t$ at each time step, and a sharp value S_v in Eq. (24) is directly added when specific situations occur ($\Gamma \neq \emptyset$), i.e.,

$$r_{R-s_i}^t = r_{sit_i}^t + S_v. \quad (27)$$

In R-s-d, the modification reward $r_{mod_i}^t$ component is removed from Eq. (20), i.e.,

$$r_{R-s-d_i}^t = r_{sit_i}^t + r_{der_i}^t. \quad (28)$$

R-s-m eliminates the $r_{der_i}^t$ term from the reward function, i.e.,

$$r_{R-s-m_i}^t = r_{sit_i}^t + r_{mod_i}^t. \quad (29)$$

Similarly, Table 3 documents SRs of the different reward variations. It is observed that ACDM exploration ability is discernibly influenced by distinct rewards. All reward variations achieve higher SRs compared to MADDPG, as they adopt ACDM frameworks, which possess the stability of behavior cloning. In case 1, R-s, R-s-d, and R-s-m attain SRs of 90.63%, 91.20% and 91.52%, making it challenging to discern performance differences. However, as the number of agents increases in case 2, there is a notable decline in SRs, which drop to 22.51%, 54.36% and 71.74%, respectively. In particular, R-s exhibits a relatively high SR in case 1 but faces difficulties in case 2. As R-s represents the most basic reward with a single term, the contrast of the two cases underscores the need for more comprehensive reward guidelines. Focusing on R-s-d and R-s-m, the anticipation is that R-s-m will outperform R-s-d. This expectation is based on the observation that $r_{der_i}^t$ solely serves as an enhancing term for $r_{sit_i}^t$, while $r_{mod_i}^t$ plays the critical role of adapting defender behaviors to accommodate different situations. The significance of $r_{mod_i}^t$ becomes more pronounced in scenarios featuring a higher number of ASVs. Conversely, in situations with fewer agents, the influence of $r_{mod_i}^t$ diminishes due to the reduced occurrence of representative scenarios. In conclusion, the proposed rewards consistently outperforms the alternative reward variations in terms of SRs in both cases.

The learning curves of the aforementioned rewards are presented in Figs. 7–8. In case 1, the curves of SRs and P_I exhibit remarkable uniformity. Moreover, the P_I curves of each reward almost overlap. Hence, the rewards under consideration have minor influences on the defense performance for solving case 1. Although the defense performances are almost the same, there are significant differences in the reward curves,

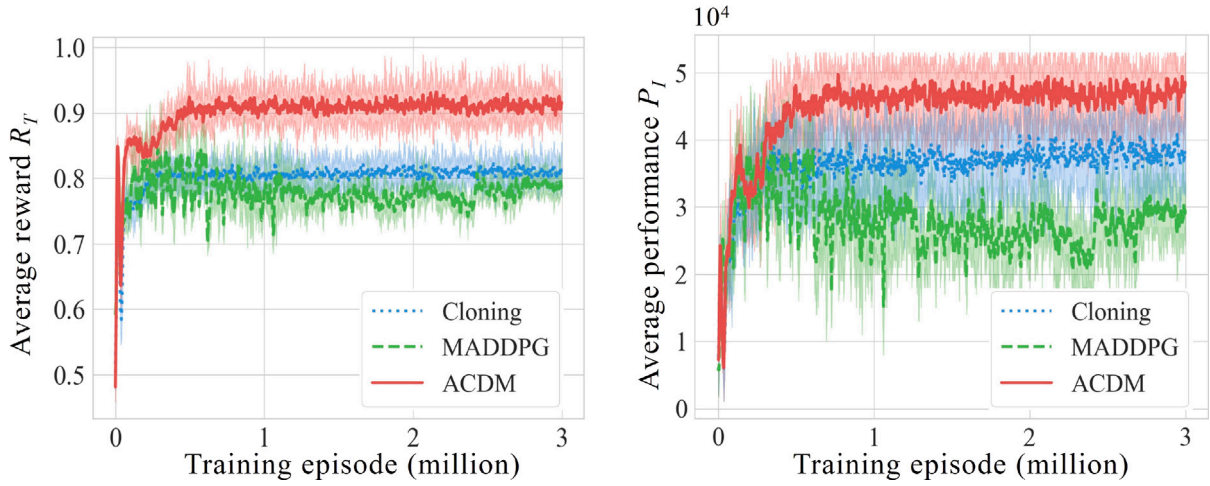


Fig. 6. Learning curves of the different methods for case 2.

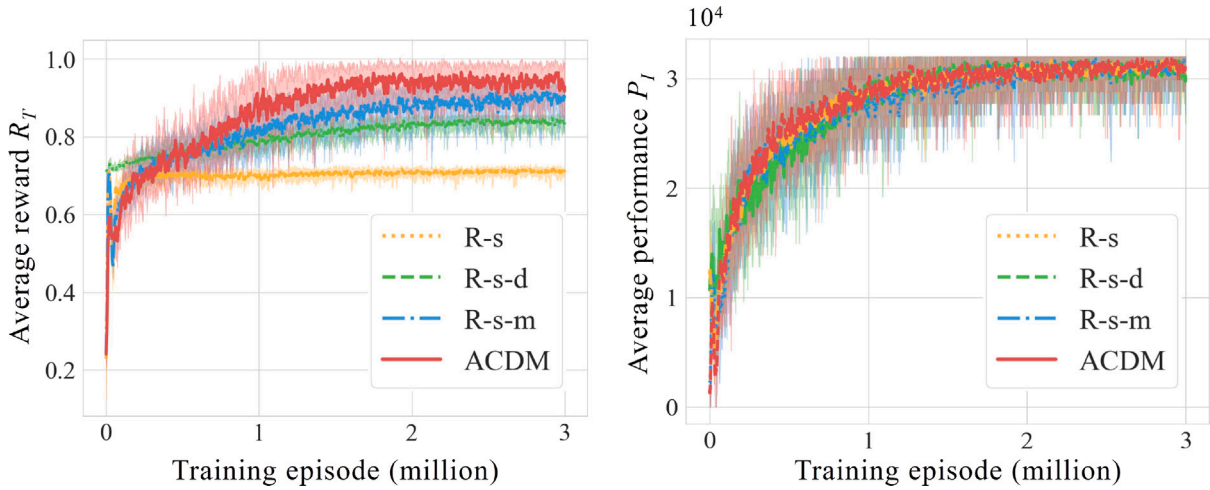


Fig. 7. Learning curves of the reward variations for case 1.

as the calculation method of rewards is different in Eqs. (20), (27)–(29). This further indicates that reward curves can only represent the learning effectiveness but cannot reflect the defense effect. In case 2, significant differences in performance come to the forefront. Notably, R-s, R-s-d, and R-s-m converge swiftly in the reward curves and trap to local optima, as indicated by P_I . For instance, R-s manages to intercept an average of merely 3–4 intruders. Consequently, it falls to achieve a high level of SRs, aligning with the results presented in Table 3. This phenomenon may be attributed to the pronounced nonstationarity inherent in the ACDS, where sharp values S_v are recurrently added to the rewards. Thus, the reward functions of R-s are multimodal and susceptible to entrapment for the employed methods. Contrarily, R-s-d exhibits the capacity to intercept approximately 4 intruders, while R-s-m demonstrates an even greater capability, intercepting 4–5 intruders. This evidence underscores the significance of incorporating each additional term within the reward function, as it consistently contributes to an enhanced defense performance. Ultimately, the proposed ACDM continues to outshine the competition in reward variations. Thus, the proposed reward in Eq. (20) incorporates the advantageous aspects of three terms, emphasizing its superiority.

4.6. Examples of specific defense processes

In this section, we present specific examples of ACDS processes, so as to provide a clearer understanding of the performance variations among the methods and the rewards in each case. It is worthy

noticing that the processes are randomly selected from the 100,000 test episodes, which can represent general cooperative defense without special handling.

Firstly, to compare the method performance, Fig. 9 shows the processes of each method in case 1. Only MADDPG fails to protect the region α_c , mainly due to being trapped in a local optima. Defenders under MADDPG control constantly rotate to expand their defensive coverage. However, they struggle to lock onto intruders efficiently, resulting in low protection efficiency and eventual failure. In contrast, defenders trained using behavior cloning quickly pursue the intruders. Nevertheless, after intercepting intruders 3 and 5, both defenders need considerable time to reorient themselves for a new target, particularly in the case of defender 2, whose trajectory is out of the designated boundary. This is because the actions provided by error-based controllers are good but not optimized. According to the control rules, an ASV needs to slow down after interceptions, leading to sharp and less smooth trajectories. On the contrary, ACDM explores more efficient policies, where ASVs can maintain high speeds and quick heading adjustment while targets are switched. As a result, the interception process of defenders controlled by ACDM are smoother and blunt, even revealing strong predictive interception capabilities.

Similarly, Fig. 10 shows the processes of each method in case 2. The target switching and turnaround issues are even more apparent in Fig. 10. For ACDM in Fig. 10(a), defenders maintain high speed and attacking precision, resulting in intruders having minimal time to

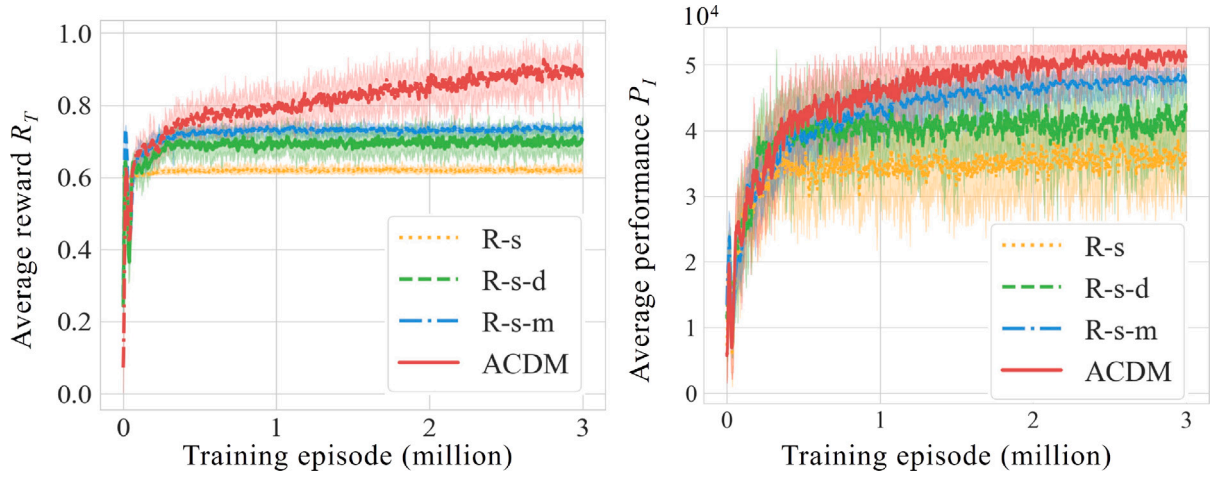


Fig. 8. Learning curves of the reward variations for case 2.

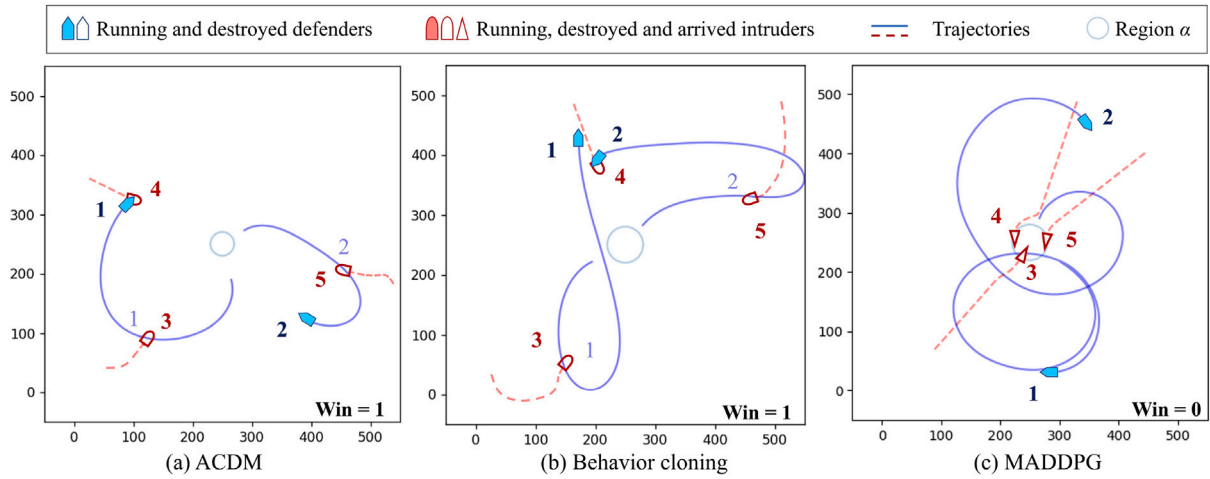


Fig. 9. Cooperative defense processes of different methods in case 1.

react and ultimately being neutralized. Thus, ACDM has the shortest trajectory and defense time, proving the efficiency. Furthermore, the attack load is distributed evenly among defenders, as their intercepted numbers are 1, 2, and 2. In the case of behavior cloning in Fig. 10(b), when a defender is close to an intruder, the actions taken are not intense due to the low errors. Consequently, intruders have the opportunity to escape, as seen with intruders 6 and 8, which successfully avoid the interception from defenders 1 and 3 to arrive at the region α_c . Thus, there is a need to explore more efficient policies based on the controller actions. On the other hand, MADDPG in Fig. 10(c), explores unique and interesting team policies, where defenders assume different roles to cooperatively complete missions. Defender 2 is allocated to sortie and attack, whereas defenders 1 and 3 are requested to stay behind to defend the region α_c . As a result, defender 2 successfully intercepts intruders 8 and 7. Defenders 1 and 3 constantly pace up and down in circle lines to extend the defense area, aiming to ensure that no defensive gaps are left behind when defenders move out to attack. Besides, intruder 5 is intercepted by defender 1, indicating that the left behind strategy has a certain defensive function. However, MADDPG is still trapped into local optimal, resulting in less efficient performance. Consequently, intruders 4 and 6 can successfully arrive at the region α_c . Although MADDPG also loses the confrontation, it showcases its ability to explore new defensive strategies.

Defense processes of the reward variations are also provided in Fig. 11, to clearly observe the motivating effects of the rewards. The reward variations have similar performance in case 1, according to

Table 3 and Fig. 7. Thus, we only provide defense processes of case 2. For R-s in Fig. 11(a), its defense process is similar but not as effective as behavior cloning in Fig. 10(b). This is because R-s only successfully intercepts 2 intruders, whereas behavior cloning intercepts 3. Moreover, two defenders are damaged by colliding in R-s. In theory, since R-s adopts ACDM framework integrating behavior cloning, R-s is expected to further explore more efficient policies than the initial policy provided by behavior cloning. However, R-s indeed less effectively than behavior cloning. This is also reflected by the statistical results in Table 3, where SRs of R-s and behavior cloning are 22.51% and 31.47% in case 2. Thus, the importance of carefully designing rewards is further highlighted here. Frequently introducing the sharp value S_v genders multimodal and high nonstationarity rewards in R-s, which not only make exploration difficult but also provide misleading guileless for defenders to learn, leading to less effective defense. For R-s-d in Fig. 11(b), it intercepts all intruders, improving attack accuracy and stability compared to behavior cloning. Defenders do not exhibit the slowdown phenomenon when closing in on intruders. Moreover, R-s-d acquire skills of attacking intruders from behind to increase SRs, as evidenced by defender 2 intercepting intruder 5 and defender 3 intercepting intruder 8. This behavior aligns with human experience regarding target acquisition. However, the slow target switching and sharp turnaround issues are not fully addressed, causing defenders 1 and 2 to run out of bounds. For R-s-m Fig. 11(c), defenders also win the confrontation. Besides, by softly revising accumulated rewards according to specific situations, R-s-m effectively addresses the slow

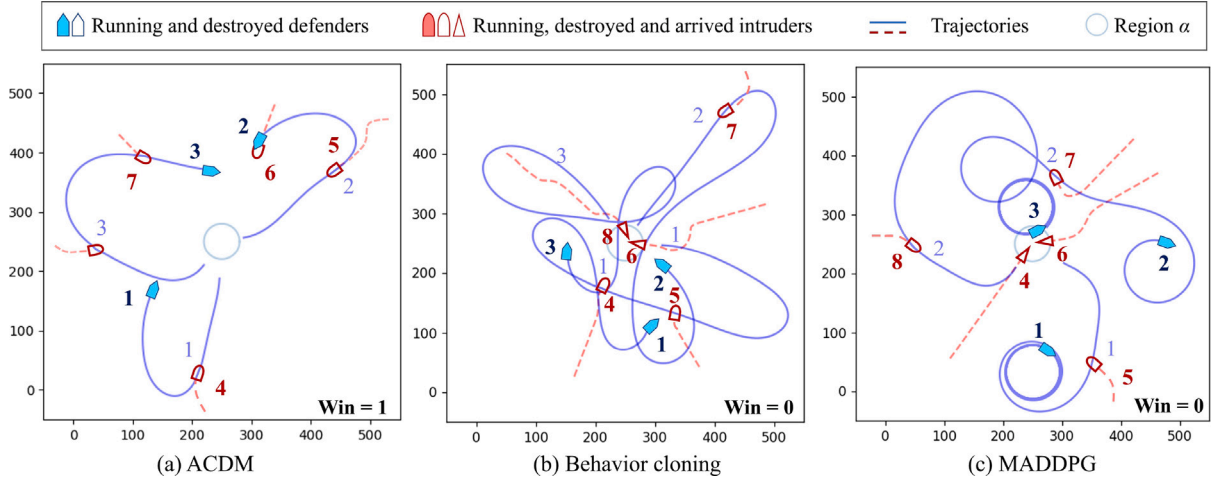


Fig. 10. Cooperative defense processes of different methods in case 2.

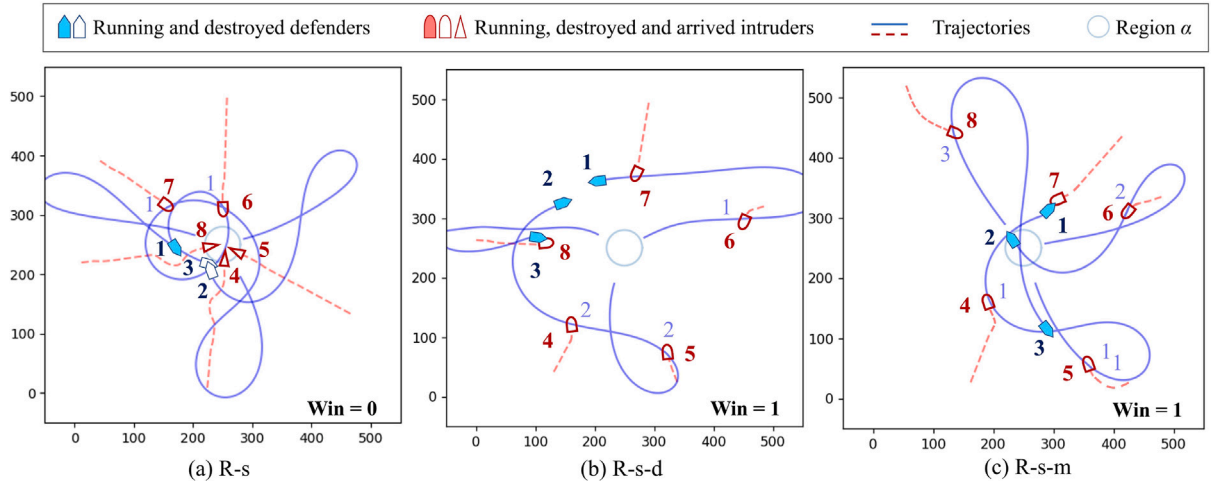


Fig. 11. Cooperative defense processes of reward variations in case 2.

target switching and sharp turnaround issues, obtaining smoother and more efficient defender movements. Finally, in the comparison of these reward variants, the superiority of ACDM over alternative rewards is evident, as shown in Fig. 9(a). Thus, each reward term in Eq. (20) has a positive impact on performance improvement, proving the effectiveness of the proposed reward design method.

In the end, the contributions and the novelty of this work can be divided into three categories: the problem to be solved, the proposed method, and the introduced reward. For the problem, unlike most defense researches, which assume defenders with numerical advantages, this paper addresses a defense mission with fewer defenders, defender damages, and intruders employing evasion strategies. However, interception challenges are also introduced, including ASV underactuated dynamics, a limited interception time window, and environmental non-stationarity, which diminish the defense success rate. For the method, most reinforcement learning methods are model-free, which overlook the deterministic information of ASV kinematics and dynamics, hindering training efficiency. Therefore, we propose a bi-level controller as the first step to generate actions, leveraging deterministic information to address the underactuated nature of ASVs. Subsequently, behavior cloning imitates the bi-level control strategy to expedite training convergence towards local optima, providing reinforcement learning with high-quality initial optimization points. Since the effectiveness of gradient-based optimizations greatly relies on the initial points, the combination of behavior cloning is very helpful for policy gradients in

reinforcement learning. This is one of the reasons for performance improvement. For the rewards, we design an innovative reward function with hybrid terms, including the situation reward $r_{sit_i}^t$, the derivative reward $r_{der_i}^t$, and the modification reward $r_{mod_i}^t$. $r_{sit_i}^t$ encourages defenders to pursue intruders effectively, while $r_{der_i}^t$ further enhances this promoting effect. However, $r_{sit_i}^t$ and $r_{der_i}^t$ do not contain the influence of interception results and the four situations. Hence, $r_{mod_i}^t$ is introduced to modify defender behavior according to the results or situations. All the terms cooperatively shape the reward R_i to align with task objectives, so that the actor network parameters μ_i can be prevented from falling into local optima.

5. Conclusions and further study

This study introduces an ASV cooperative defense scenario, referred to as ACDS, characterized by a reduced number of defenders, potential defender damages, and intruder evasion strategies. Traditional methods face difficulties in solving ACDS due to the underactuated dynamics of ASVs, learning nonstationarity, and a limited interception time window. To surmount these challenges, we combine the strengths of behavior cloning and deep reinforcement learning. Initially, we design a bi-level controller specifically tailored to generate ASV actions, which address the inherent underactuated nature. Besides, these controller-generated actions are then employed in conjunction with behavior cloning, so that ASVs can imitate the actions to accelerate the training convergence towards local optima. Subsequently, our approach

leverages a deep reinforcement learning algorithm to explore and refine more efficient policies, so as to ensure the successful protection within the limited interception time window. Furthermore, we design rewards with hybrid terms to effectively drive policy optimizations, while also mitigating the inherent nonstationarity in the environment. Through a series of numerical simulations across two ACDS cases, we rigorously evaluate the effectiveness of our approach. Specifically, our method consistently achieves the highest success rates compared to behavior cloning and a well used multi-agent reinforcement learning method, namely MADDPG. Besides, the proposed reward design also demonstrates superiority among the three other reward variations. Additionally this research not only uncovers the performance limitations of behavior cloning, but also underscores the exploration capabilities of reinforcement learning. Moreover, we analyze the impacts of different reward variations through systematic comparisons.

The proposed method is limited to effectively solve scenarios involving a smaller number of ASVs. Additionally, the defenders are equipped with the same abilities, which are isomorphic. However, small-scale homogeneous ASV groups may face bottlenecks, such as lacks of flexibility and defense capabilities. In future research, we intend to investigate cooperative defense mission of large-scale heterogeneous ASV clusters, where the price, motion ability, and equipped weapons of defensive ASVs all vary. However, autonomously making decisions in such mission is more challenging. Firstly, as the number of ASVs increases, the curse of dimensionality in the states and the action space becomes a notable concern. Secondly, the differences in ASV properties and functionalities require high-quality task allocation abilities. For example, faster ASVs are better suited for sortie interception, while slower ASVs are more effective for waiting to supplement defense gaps. Finally, the complex decision process need to consider more multiple mission objectives, such as interception time and costs, introducing challenges in policy exploration. To address the these, the proposed ACMD can be improved by following aspects: Firstly, dimensionality reduction techniques, such as parameter sharing or mean field methods, can be implemented to mitigate the curse of dimensionality in the state and action space. Secondly, Nash equilibrium strategies can be used to generate actions for behavior cloning, enabling optimal mission allocation. Finally, evolutionary reinforcement learning, which integrates metaheuristics and deep reinforcement learning, can be employed as the training method under multiple mission objectives. These approaches offer stronger policy exploration capabilities, allowing for more effective decision-making in complex scenarios.

CRedit authorship contribution statement

Siqing Sun: Writing – review & editing, Writing – original draft, Validation, Software, Methodology, Investigation, Conceptualization. **Tianbo Li:** Writing – review & editing, Validation, Supervision, Conceptualization. **Xiao Chen:** Software, Investigation, Conceptualization. **Huachao Dong:** Supervision, Resources, Funding acquisition. **Xinjing Wang:** Writing – review & editing, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgments

Supports from the National Natural Science Foundation of China (Grant No. 52175251) and (Grant No. 52205268), and the National Basic Scientific Research Program of China (Grant No. JCKY2021206B005) are gratefully acknowledged.

References

- [1] B. Ning, Q.-L. Han, Z. Zuo, L. Ding, Q. Lu, X. Ge, Fixed-time and prescribed-time consensus control of multiagent systems and its applications: A survey of recent trends and methodologies, *IEEE Trans. Ind. Inform.* 19 (2) (2023) 1121–1135, <http://dx.doi.org/10.1109/TII.2022.3201589>.
- [2] W. Chen, C. Gao, W. Jing, Proximal policy optimization guidance algorithm for intercepting near-space maneuvering targets, *Aerosp. Sci. Technol.* 132 (2023) 108031.
- [3] R. Lowe, Y. Wu, A. Tamar, J. Harb, O. Pieter Abbeel, I. Mordatch, Multi-agent actor-critic for mixed cooperative-competitive environments, in: *Advances in Neural Information Processing Systems*, Vol. 30, 2017, pp. 6382–6393.
- [4] E. Raboin, P. Švec, D.S. Nau, S.K. Gupta, Model-predictive asset guarding by team of autonomous surface vehicles in environment with civilian boats, *Auton. Robots* 38 (2015) 261–282.
- [5] X. Meng, B. Sun, D. Zhu, Harbour protection: moving invasion target interception for multi-AUV based on prediction planning interception method, *Ocean Eng.* 219 (2021) 108268.
- [6] Y. Yu, J. Liu, C. Wei, Hawk and pigeon's intelligence for UAV swarm dynamic combat game via competitive learning pigeon-inspired optimization, *Sci. China Technol. Sci.* 65 (5) (2022) 1072–1086.
- [7] M. Tan, H. Shen, Three-dimensional cooperative game guidance law for a leader-follower system with impact angles constraint, *IEEE Trans. Aerosp. Electron. Syst.* 60 (1) (2024) 405–420, <http://dx.doi.org/10.1109/TAES.2023.3325795>.
- [8] S. Sui, S. Tong, Finite-time fuzzy adaptive PPC for nonstrict-feedback nonlinear MIMO systems, *IEEE Trans. Cybern.* 53 (2) (2023) 732–742, <http://dx.doi.org/10.1109/TCYB.2022.3163739>.
- [9] T. Kang, N. Gu, D. Wang, L. Liu, Q. Hu, Z. Peng, Neurodynamics-based attack-defense guidance of autonomous surface vehicles against multiple attackers for domain protection, *IEEE Trans. Ind. Electron.* (2024) 1–9, <http://dx.doi.org/10.1109/TIE.2024.3360603>.
- [10] Y. Rizk, M. Awad, E.W. Tunstel, Decision making in multiagent systems: A survey, *IEEE Trans. Cogn. Dev. Syst.* 10 (3) (2018) 514–529.
- [11] F. Khemakhem, H. Ellouzi, H. Ltfi, M.B. Ayed, Agent-based intelligent decision support systems: A systematic review, *IEEE Trans. Cogn. Dev. Syst.* 14 (1) (2022) 20–34.
- [12] S. Sui, C.L.P. Chen, S. Tong, A novel full errors fixed-time control for constraint nonlinear systems, *IEEE Trans. Autom. Control* 68 (4) (2023) 2568–2575, <http://dx.doi.org/10.1109/TAC.2022.3200962>.
- [13] Z. Peng, X. Song, S. Song, V. Stojanovic, Hysteresis quantified control for switched reaction-diffusion systems and its application, *Complex Intell. Syst.* (2023) 1–10, <http://dx.doi.org/10.1007/s40747-023-01135-y>.
- [14] C. Liu, S. Sun, C. Tao, Y. Shou, B. Xu, Sliding mode control of multi-agent system with application to UAV air combat, *Comput. Electr. Eng.* 96 (2021) 107491.
- [15] H. Duan, J. Zhao, Y. Deng, Y. Shi, X. Ding, Dynamic discrete pigeon-inspired optimization for multi-UAV cooperative search-attack mission planning, *IEEE Trans. Aerosp. Electron. Syst.* 57 (1) (2020) 706–720.
- [16] B. Wang, S. Li, X. Gao, T. Xie, Weighted mean field reinforcement learning for large-scale UAV swarm confrontation, *Appl. Intell.* 53 (5) (2023) 5274–5289.
- [17] N. Chen, L. Li, W. Mao, Equilibrium strategy of the pursuit-evasion game in three-dimensional space, *IEEE/CAA J. Autom. Sin.* 11 (2) (2024) 446–458.
- [18] T. Duan, W. Wang, T. Wang, A review for unmanned swarm gaming: Framework, model and algorithm, in: *2022 8th International Conference on Big Data and Information Analytics, BigDIA, IEEE*, 2022, pp. 164–170.
- [19] F. Liu, X. Dong, J. Yu, Y. Hua, Q. Li, Z. Ren, Distributed Nash equilibrium seeking of N -coalition noncooperative games with application to UAV swarms, *IEEE Trans. Netw. Sci. Eng.* 9 (4) (2022) 2392–2405.
- [20] E. Antonioni, V. Suriani, F. Riccio, D. Nardi, Game strategies for physical robot soccer players: a survey, *IEEE Trans. Games* 13 (4) (2021) 342–357.
- [21] S. Sun, B. Song, P. Wang, H. Dong, X. Chen, Real-time mission-motion planner for multi-UUVs cooperative work using tri-level programming, *IEEE Trans. Intell. Transp. Syst.* 23 (2) (2022) 1260–1273, <http://dx.doi.org/10.1109/TITS.2020.3023819>.
- [22] T. Li, S. Sun, P. Wang, H. Dong, X. Wang, A multi-objective bi-level task planning strategy for UUV target visitation in ocean environment, *Ocean Eng.* 288 (2023) 116022.
- [23] Y. Lei, M. Huo, Y. Deng, H. Duan, Multiple UAVs target allocation via stochastic dominant learning pigeon-inspired optimization in beyond-visual-range air combat, in: *2022 12th International Conference on CYBER Technology in Automation, Control, and Intelligent Systems, CYBER, IEEE*, 2022, pp. 1269–1274.
- [24] J. Cai, F. Zhang, S. Sun, T. Li, A meta-heuristic assisted underwater glider path planning method, *Ocean Eng.* 242 (2021) 110121.
- [25] C. Chen, X. Wang, H. Dong, P. Wang, Surrogate-assisted hierarchical learning water cycle algorithm for high-dimensional expensive optimization, *Swarm Evol. Comput.* 75 (2022) 101169.

- [26] P. Cheng, H. Wang, V. Stojanovic, F. Liu, S. He, K. Shi, Dissipativity-based finite-time asynchronous output feedback control for wind turbine system via a hidden Markov model, *Int. J. Syst. Sci.* 53 (15) (2022) 3177–3189.
- [27] V. Mnih, K. Kavukcuoglu, D. Silver, A.A. Rusu, J. Veness, M.G. Bellemare, A. Graves, M. Riedmiller, A.K. Fidjeland, G. Ostrovski, et al., Human-level control through deep reinforcement learning, *Nature* 518 (7540) (2015) 529–533.
- [28] D. Silver, J. Schrittwieser, K. Simonyan, I. Antonoglou, A. Huang, A. Guez, T. Hubert, L. Baker, M. Lai, A. Bolton, et al., Mastering the game of go without human knowledge, *Nature* 550 (7676) (2017) 354–359.
- [29] O. Vinyals, I. Babuschkin, W.M. Czarnecki, M. Mathieu, A. Dudzik, J. Chung, D.H. Choi, R. Powell, T. Ewalds, P. Georgiev, et al., Grandmaster level in StarCraft II using multi-agent reinforcement learning, *Nature* 575 (7782) (2019) 350–354.
- [30] Y. Li, W. Han, Y. Wang, Deep reinforcement learning with application to air confrontation intelligent decision-making of manned/unmanned aerial vehicle cooperative system, *IEEE Access* 8 (2020) 67887–67898, <http://dx.doi.org/10.1109/ACCESS.2020.2985576>.
- [31] B. Li, J. Wang, C. Song, Z. Yang, K. Wan, Q. Zhang, Multi-UAV roundup strategy method based on deep reinforcement learning CEL-MADDPG algorithm, *Expert Syst. Appl.* 245 (2024) 123018.
- [32] Y. Hu, X. Wang, Y. Liu, W. Ding, A. Knoll, PI-ELM: Reinforcement learning-based adaptable policy improvement for dynamical system, *Inform. Sci.* 650 (2023) 119700.
- [33] T. Hester, M. Vecerik, O. Pietquin, M. Lanctot, T. Schaul, B. Piot, D. Horgan, J. Quan, A. Sendonaris, I. Osband, et al., Deep q-learning from demonstrations, in: *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 32, 2018, pp. 3223–3230.
- [34] Z. Huang, J. Wu, C. Lv, Efficient deep reinforcement learning with imitative expert priors for autonomous driving, *IEEE Trans. Neural Netw. Learn. Syst.* 34 (10) (2023) 7391–7403, <http://dx.doi.org/10.1109/TNNLS.2022.3142822>.
- [35] J. Wang, P. Zhang, Y. Wang, Autonomous target tracking of multi-UAV: A two-stage deep reinforcement learning approach with expert experience, *Appl. Soft Comput.* 145 (2023) 110604.
- [36] Z. Wang, F. Liu, J. Guo, C. Hong, M. Chen, E. Wang, Y. Zhao, UAV swarm confrontation based on multi-agent deep reinforcement learning, in: *2022 41st Chinese Control Conference, CCC*, 2022, pp. 4996–5001, <http://dx.doi.org/10.23919/CCC55666.2022.9902663>.
- [37] H. Zhou, X. Zhang, Z. Zhang, Reinforcement learning technology for air combat confrontation of unmanned aerial vehicle, in: *International Conference on Computer Graphics, Artificial Intelligence, and Data Processing, ICCAID 2021*, Vol. 12168, SPIE, 2022, pp. 454–459.
- [38] K. Wan, D. Wu, B. Li, X. Gao, Z. Hu, D. Chen, ME-MADDPG: An efficient learning-based motion planning method for multiple agents in complex environments, *Int. J. Intell. Syst.* 37 (3) (2022) 2393–2427.
- [39] T.T. Nguyen, N.D. Nguyen, S. Nahavandi, Deep reinforcement learning for multiagent systems: A review of challenges, solutions, and applications, *IEEE Trans. Cybern.* 50 (9) (2020) 3826–3839.
- [40] T. Rupprecht, Y. Wang, A survey for deep reinforcement learning in markovian cyber-physical systems: Common problems and solutions, *Neural Netw.* 153 (2022) 12–36.
- [41] L. Huang, M. Fu, H. Qu, S. Wang, S. Hu, A deep reinforcement learning-based method applied for solving multi-agent defense and attack problems, *Expert Syst. Appl.* 176 (2021) 114896.
- [42] B. Wang, S. Li, X. Gao, T. Xie, UAV swarm confrontation using hierarchical multiagent reinforcement learning, *Int. J. Aerosp. Eng.* 2021 (2021) 3360116.
- [43] S. Xuan, L. Ke, UAV swarm attack-defense confrontation based on multi-agent reinforcement learning, in: *Advances in Guidance, Navigation and Control*, in: *Lecture Notes in Electrical Engineering*, 2022, pp. 5599–5608, http://dx.doi.org/10.1007/978-981-15-8155-7_464.
- [44] T. Zhang, L. Chai, S. Wang, J. Jin, X. Liu, A. Song, Y. Lan, Improving autonomous behavior strategy learning in an unmanned swarm system through knowledge enhancement, *IEEE Trans. Reliab.* 71 (2) (2022) 763–774.
- [45] B.-B. Hu, H.-T. Zhang, J. Wang, Multiple-target surrounding and collision avoidance with second-order nonlinear multiagent systems, *IEEE Trans. Ind. Electron.* 68 (8) (2021) 7454–7463, <http://dx.doi.org/10.1109/TIE.2020.3000092>.
- [46] C. Tang, H.-T. Zhang, S. Sun, Hierarchical elastic formation control for multiple autonomous surface vessels, in: *2023 China Automation Congress, CAC*, 2023, pp. 8777–8781, <http://dx.doi.org/10.1109/CAC59555.2023.10450957>.
- [47] C. Tang, H.-T. Zhang, J. Wang, Flexible formation tracking control of multiple unmanned surface vessels for navigating through narrow channels with unknown curvatures, *IEEE Trans. Ind. Electron.* 70 (3) (2023) 2927–2938, <http://dx.doi.org/10.1109/TIE.2022.3169825>.
- [48] M. Lauri, D. Hsu, J. Pajarinen, Partially observable Markov decision processes in robotics: A survey, *IEEE Trans. Robot.* 39 (1) (2023) 21–40.
- [49] B. Liu, Z. Chen, H.-T. Zhang, X. Wang, T. Geng, H. Su, J. Zhao, Collective dynamics and control for multiple unmanned surface vessels, *IEEE Trans. Control Syst. Technol.* 28 (6) (2019) 2540–2547.
- [50] R. Zhang, Q. Zong, X. Zhang, L. Dou, B. Tian, Game of drones: Multi-UAV pursuit-evasion game with online motion planning by deep reinforcement learning, *IEEE Trans. Neural Netw. Learn. Syst.* 34 (10) (2023) 7900–7909, <http://dx.doi.org/10.1109/TNNLS.2022.3146976>.